Algorithms

Instructor:

Tami Tamir

TAs:

Daniel Prigan

Yael Berkman

Daniel Karalnik

Graders:

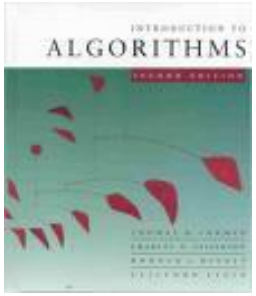
Daniel Karalnik

Liam Tal



General

- Textbook:
  - Cormen, Leiserson, Rivest and Stein
    Introduction to Algorithms



- Final grade components:
  - H = average of best 7 (out of 8) HWs
  - M= Mid-term exam
  - E = Final exam
- Final grade calculation:
  - If (E<60) then FAIL (sorry).
  - Else 0.10*min(H, E+25)
    + 0.1*M + 0.6*E + 0.15*max(M,E)



Resources

- Moodle – lecture notes, HWs, all files
- Piazza – forum
- Panopto - recordings
- Mentimeter
- Many reception hours. Please come!



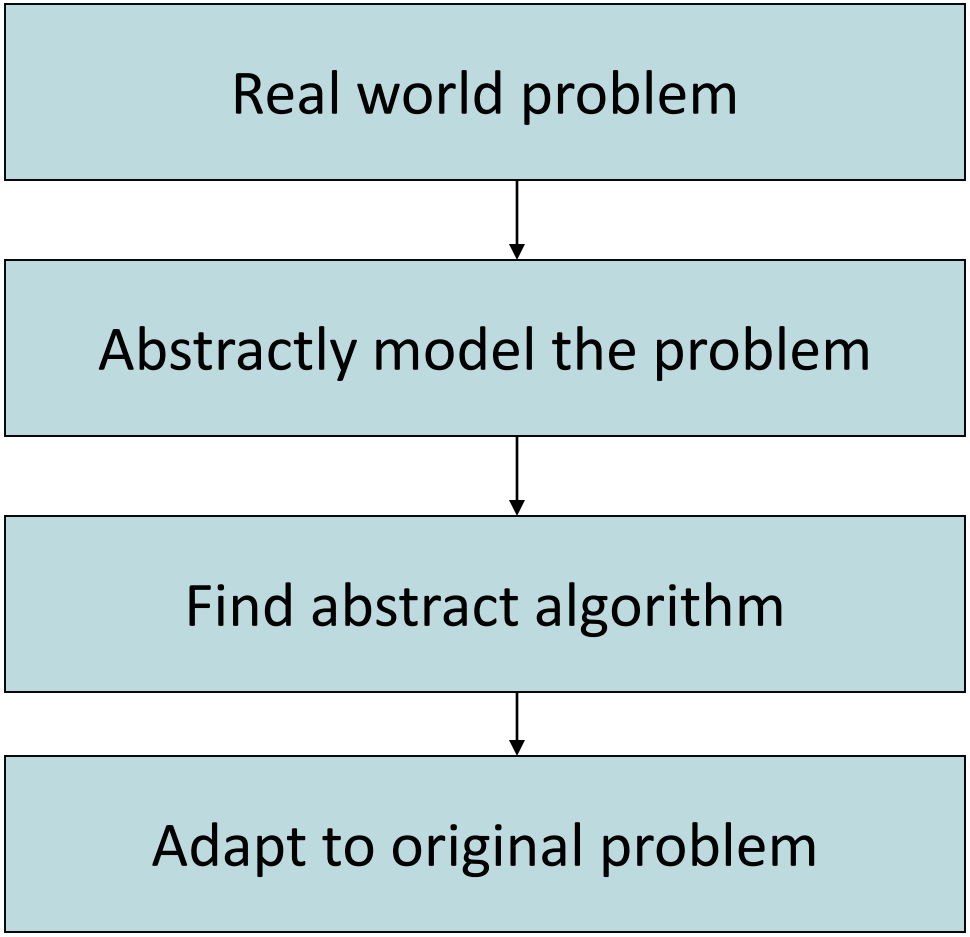
Course Goals



- A better sense of how to model problems you encounter as well-known algorithmic problems.
- A deeper understanding of the issues and tradeoffs involved in algorithm design.
- A large toolbox – with classical algorithms, concepts, and techniques.
- An appreciation for applications of algorith techniques in the real world.
- Fun!! Beauty!!!



Applied Algorithm Scenario



Modeling

- What kind of algorithm is needed?
- Can I find an algorithm, or do I have to invent one?
- Can I ‘tune’ an existing algorithm? Does it remind me a familiar problem?

Algorithm Design Goals

- Correctness
- Efficiency
- Simple, if possible.

Types of complexity

- We should analyze separately ‘good’ and ‘bad’ inputs.
- Processing time is surely a bad measure.
- We need a ‘stable’ measure, independent of the implementation.

- A complexity function is a function T: N → N.

  T(n) is the number of operations the algorithm does on an input of size n.

- We can measure three different things.

- Worst-case complexity
- Best-case complexity
- Average-case complexity

The RAM Model of Computation

- Each simple operation takes 1 time step.
- Loops and subroutines are not simple operations.
- Each memory access takes one time step, and there is no shortage of memory.

For a given problem instance:

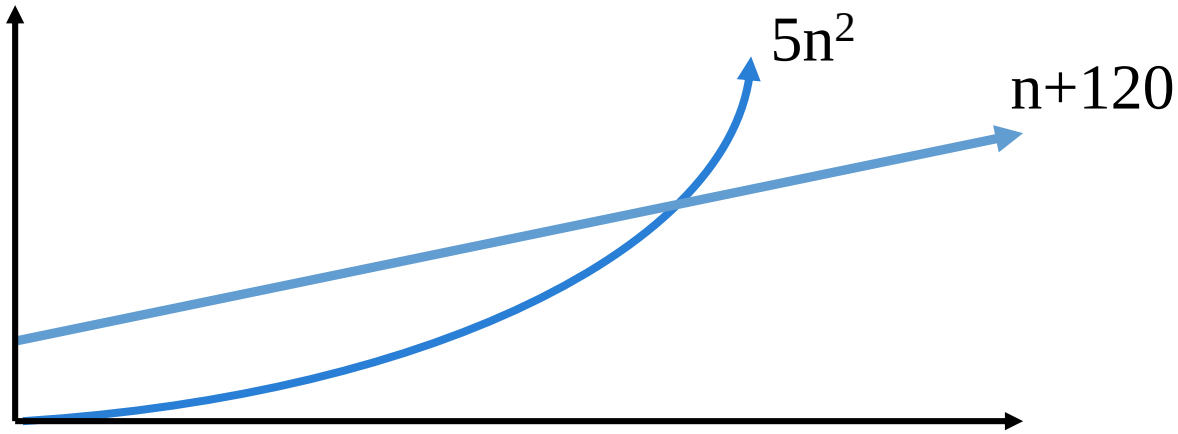
- Running time of an algorithm = # RAM steps.
- Space used by an algorithm = # RAM memory cells

This is a useful abstraction. It allows us to analyze algorithms in a machine independent fashion.

Big O Notation

- Goal :
  - A stable measurement independent of the machine.
- Way:
  - ignore constant factors.
- f(n) = O(g(n)) if c·g(n) is upper bound on f(n)

$\Leftrightarrow$ There exist c, N, s.t. for any $n \geq N$, $f(n) \leq c \cdot g(n)$

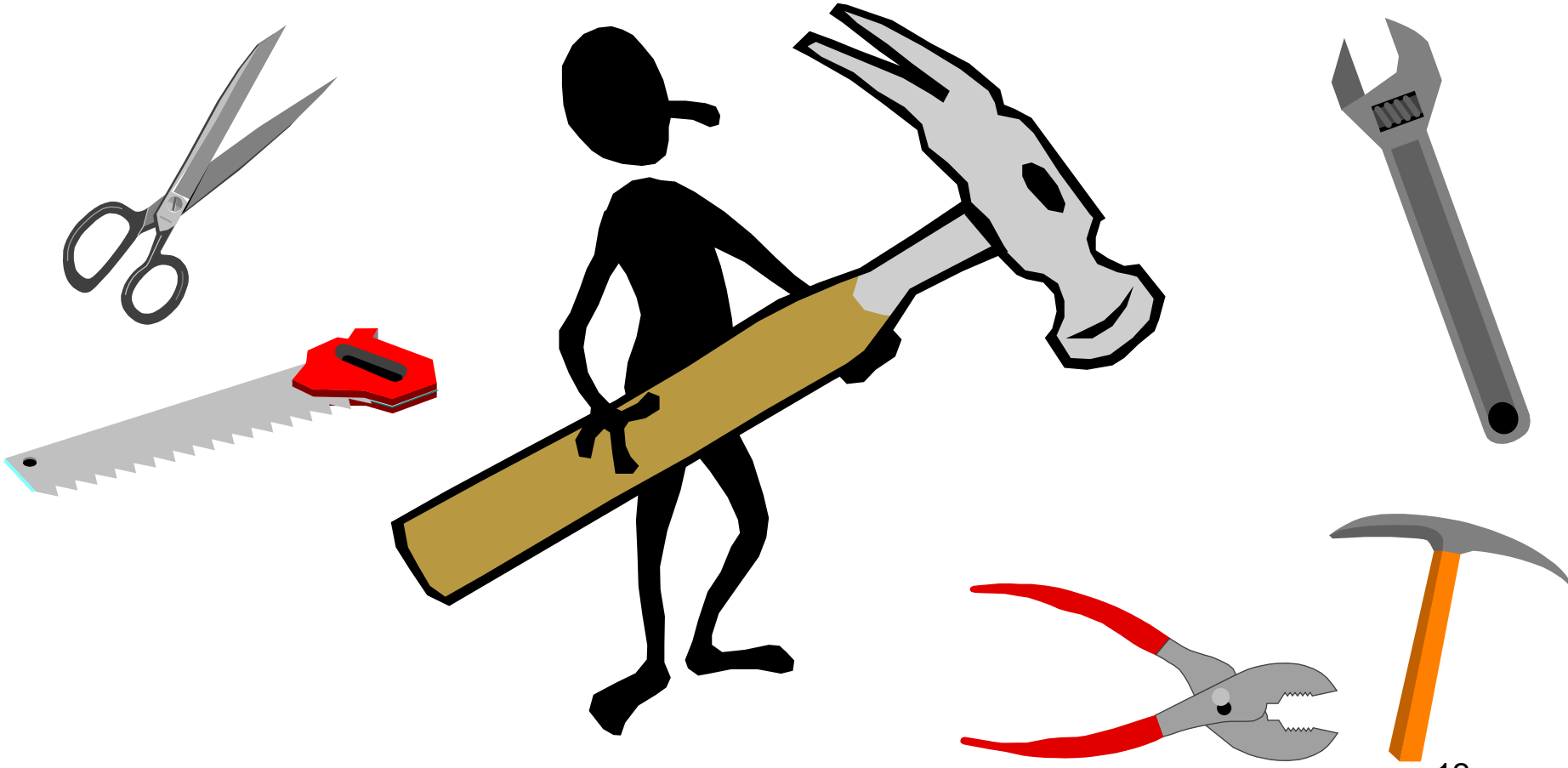


For all $n \geq 120$, $n+120 \leq 2n \Rightarrow n+120 = O(n)$

Growth Rates

- Even by ignoring constant factors, we can get an excellent idea of whether a given algorithm will be able to run in a reasonable amount of time on a problem of a given size.
- The “big O” notation and worst-case analysis are tools that greatly simplify our ability to compare the efficiency of algorithms.

Your goal: Build a toolbox of abstract structures and concepts. Know the capacities and limits of each tool.

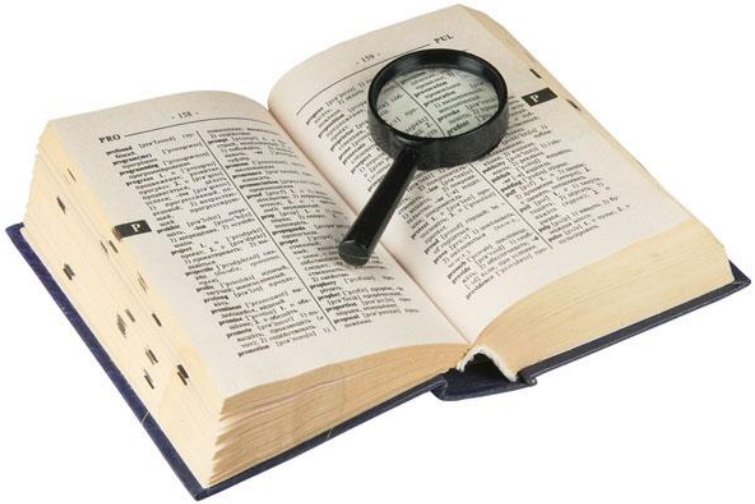


Appreciate Alternative Solutions To The Same Problem

When presented with multiple solutions don't remember only the one that you find easiest to understand. The expert learner takes the opportunity to think through the similarities and differences between the approaches. Such meditations lay the foundations for effective learning and problem solving.

Example: Anagram Programming Task

You are given a 70,000-word dictionary. Write an anagram utility that given a word as input returns all anagrams of that word appearing in the dictionary.



Examples

- Input: CAT
- Output: ACT, CAT, TAC

- Input: SUBESSENTIAL
- Output: SUITABLENESS

Solution 1: Impatient Hacker

- Loop through all possible ways of rearranging the input word
  - Use binary search to look up resulting word in dictionary.
  - If found, output it.

Performance Analysis:

On input “microphotographic” the loop will run $17! >> 3 * 10^{14}$ iterations.

Even at one microsecond (10^{-6}) per iteration such a loop takes $3*10^{8}$ seconds.

This is about a decade.

Solution 2: “Expert” Hacker

- Subroutine ANAGRAM(X,Y) returns **TRUE** if X and Y are anagrams. It works by sorting the letters in X and Y

- Input X
- Loop through all dictionary words Y
  - If ANAGRAM(X,Y) output Y

Comparing an input word with each of 70,000 dictionary entries takes several seconds.

The hacker is satisfied

Solution 3: The master



- The master's program will have the minimal possible time complexity.

- What is the minimal possible time complexity?

Master's Solution

- Don't keep the dictionary in sorted order!

- Rearranging the dictionary into anagram classes will make the original problem simple.

Suppose the dictionary is the list below.

- ASP
- DOG
- LURE
- GOD
- NICE
- RULE
- SPA

After each word, write its "signature" (sort its letters)

- ASP APS
- DOG DGO
- LURE ELRU
- GOD DGO
- NICE CEIN
- RULE ELRU
- SPA APS

Sort by the signatures

- ASP APS
- SPA APS
- NICE CEIN
- DOG DGO
- GOD DGO
- LURE ELRU
- RULE ELRU

Master's Program

- Input word W (e.g., CAT)
- X := signature of W (ACT)
- Use binary search to find the anagram class of W and output it.

About $\log_2(70000) \times 25$ microseconds
$\approx .0004$ seconds.

Comparing the three solutions

- For a dictionary of n words, input word W, and Z output words.

Solution 1: $O(|W|! \cdot \log n)$

Solution 2: $O(n|W|)$

Solution 3: $O(\log(n) + Z|W|)$.

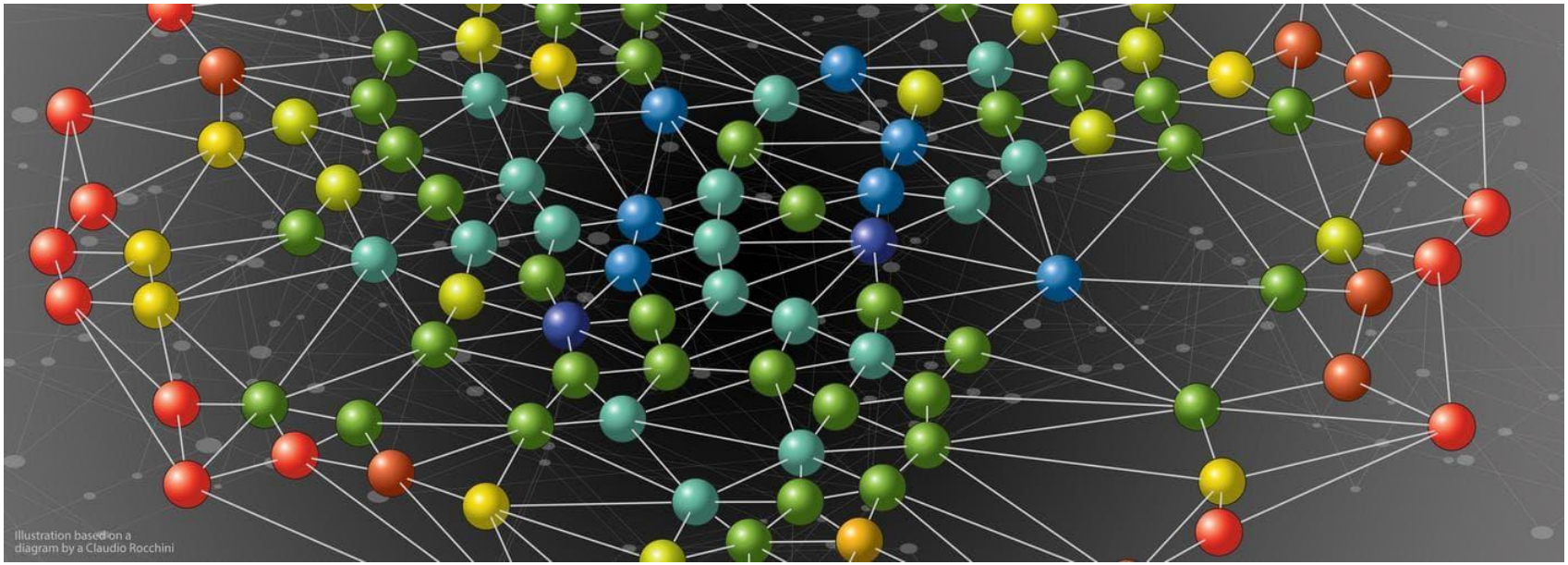
Time of solution 3: Output size + One Find operation. We cannot expect to do significantly better (can reduce the O(log n) to O(1) using a Hash-table for the signatures).

Right, it takes about 20 seconds to create the dictionary, but it is perfectly fair to think of this as programming time. The building of the dictionary is a one-time cost that is part of writing the program.

Add to toolbox: Preprocessing

- It is sometimes possible to pay a reasonable, one-time preprocessing cost to reorganize your data in such a way as to use it more efficiently later. The extra time required to preprocess can be thought of as additional programming effort.

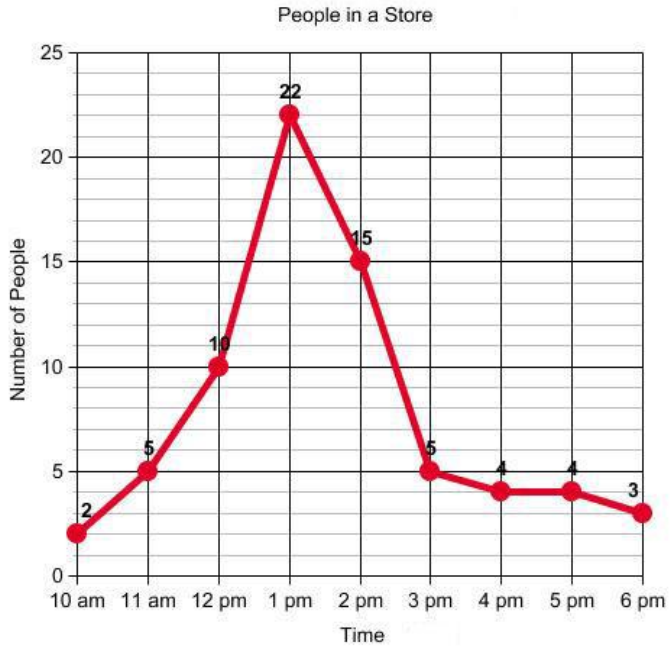
Get ready for our main topic…



GRAPH ALGORITHMS!!!!

Graph Algorithms

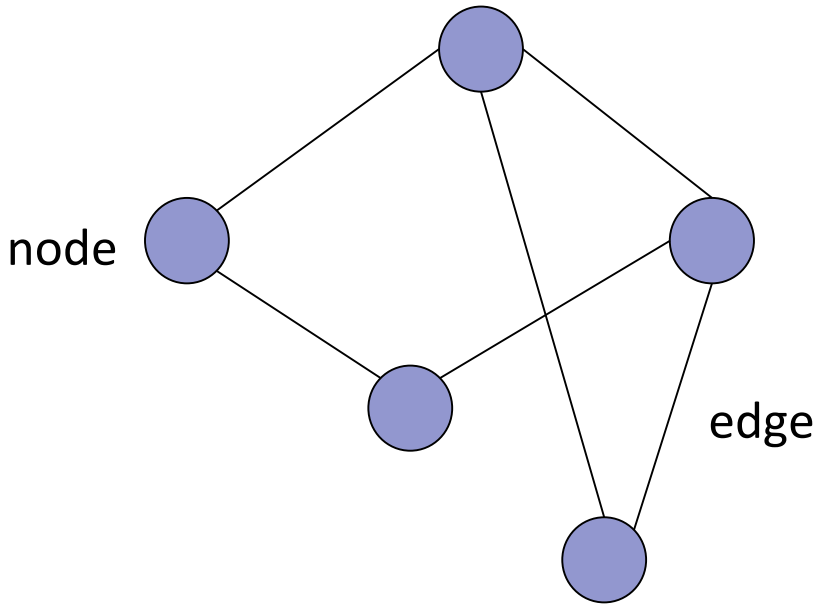
- What are graphs????
- Yes, this is a graph….



- But we are interested in a different kind of “graph”

Graphs

- Graphs consist of
  - Nodes (vertices)
  - Edges (arcs)

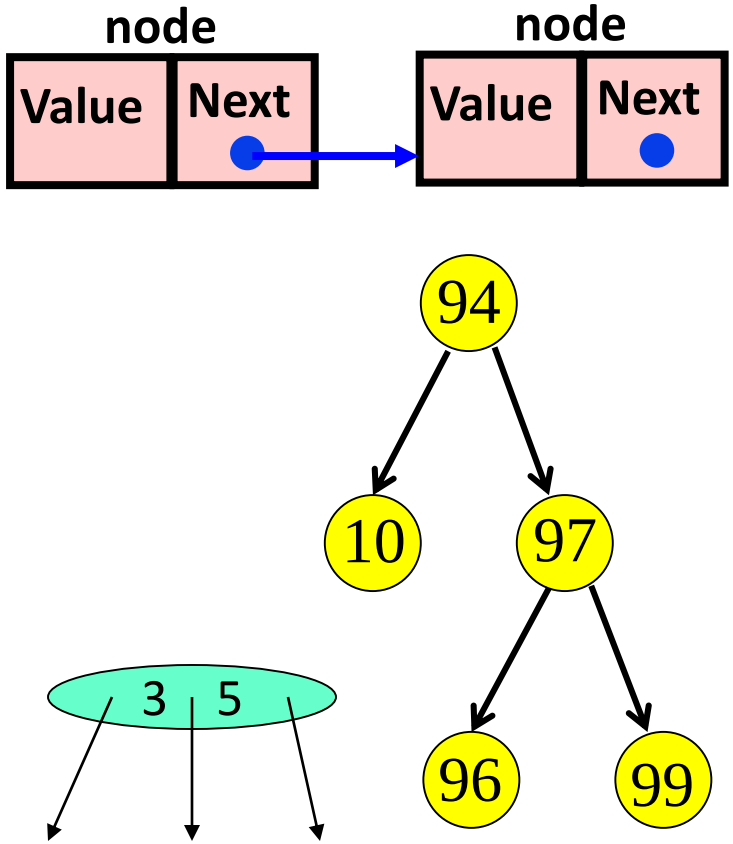


Variants of Graphs

- Nodes
  - Labeled or unlabeled
- Edges
  - Directed or undirected
  - Labeled or unlabeled

Motivation for Graphs

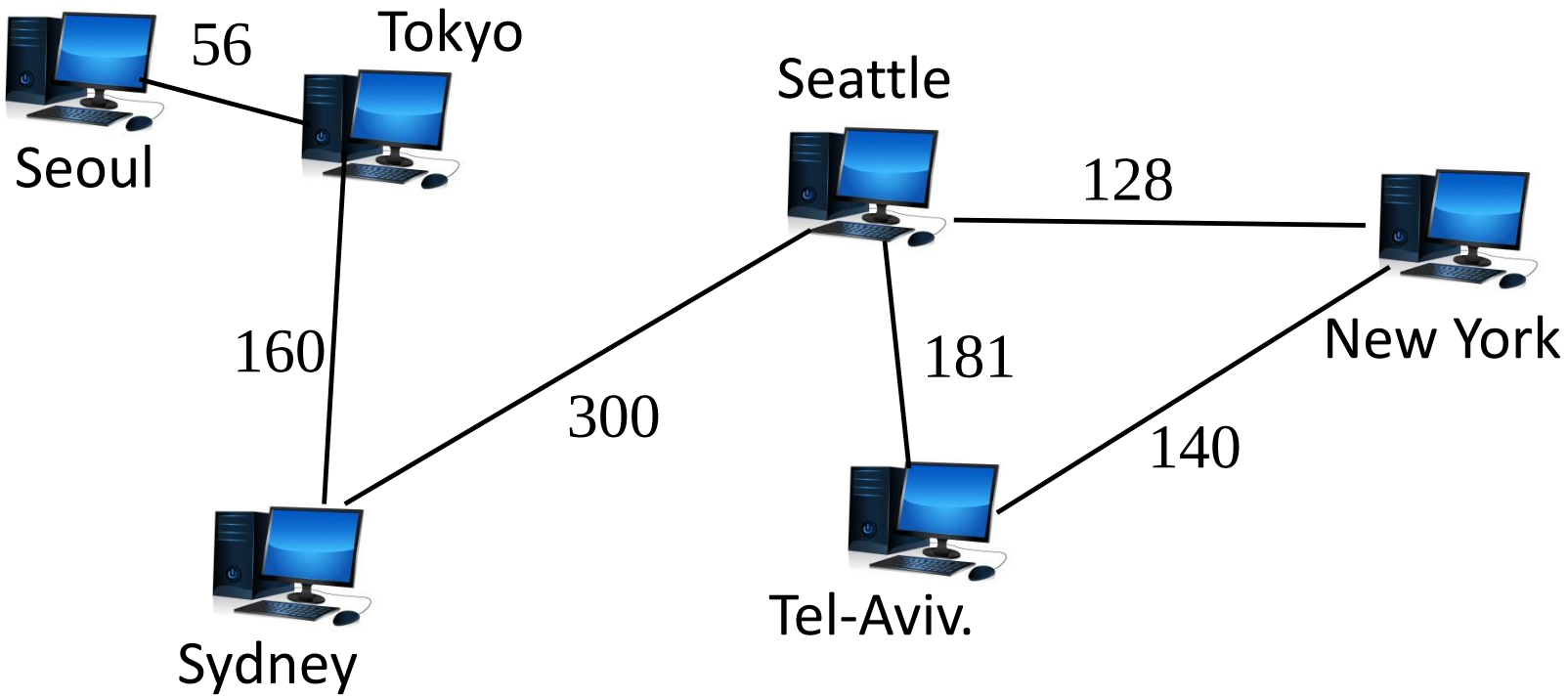
- Consider the data structures we have looked at so far…
- Linked list: nodes with 1 incoming edge + 1 outgoing edge
- Binary trees/heaps: nodes with 1 incoming edge + 2 outgoing edges
- B-trees: nodes with 1 incoming edge + multiple outgoing edges



Motivation for Graphs

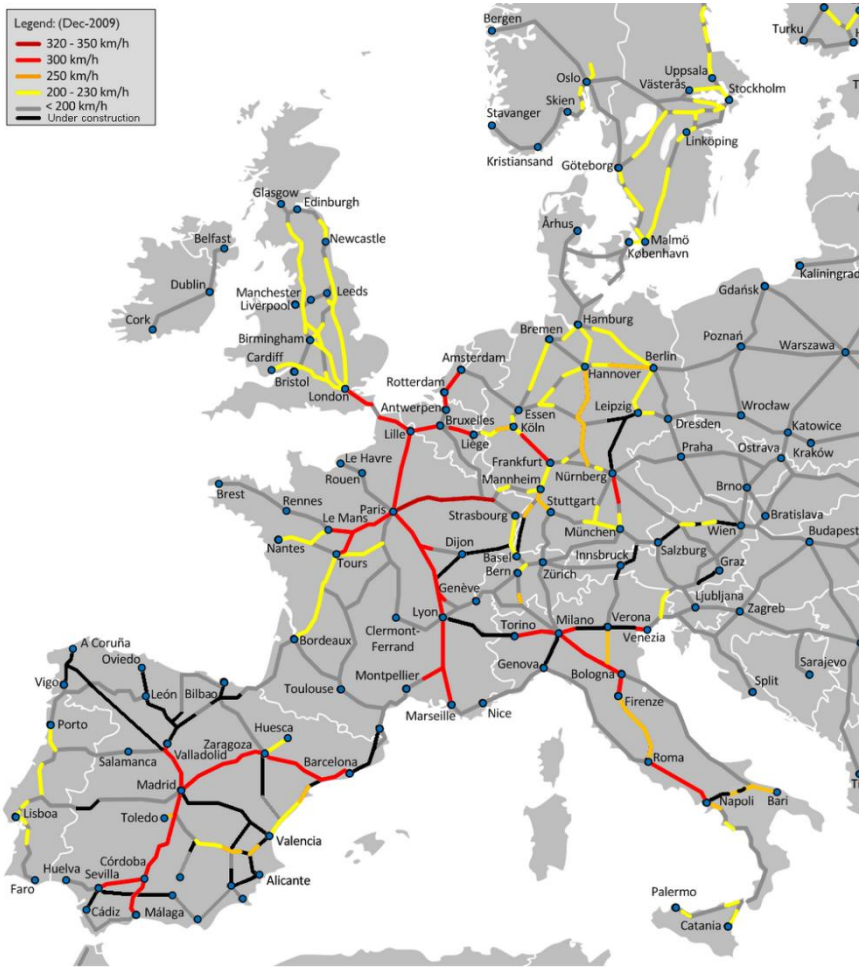
- How can you generalize these data structures?
- Graphs can be used to model connections between any type of objects.
- Here are some examples:

Information Transmission in a Computer Network



Nodes = computers
Edges = transmission rates

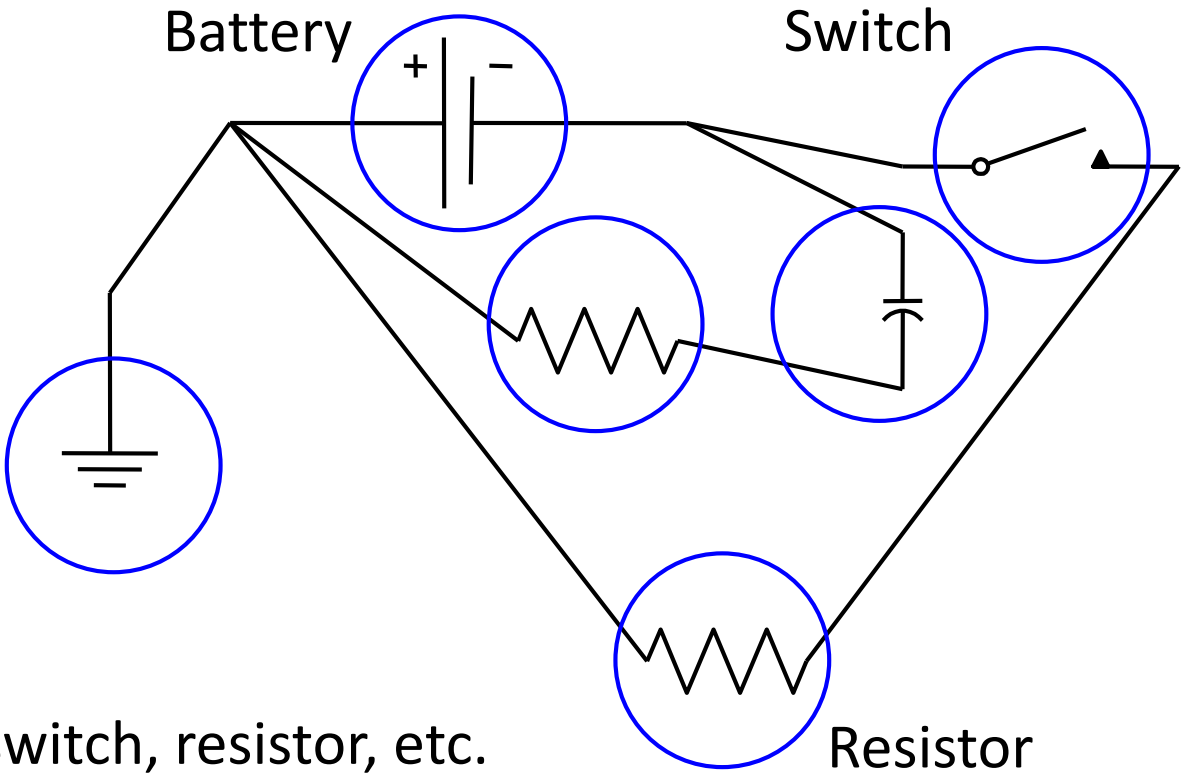
Geographical map



Nodes = cities
Edges = connecting highways.

Can be used to model traffic flow, etc.

Representing Electrical Circuits



Nodes = battery, switch, resistor, etc.
Edges = connections

Precedence-constraints

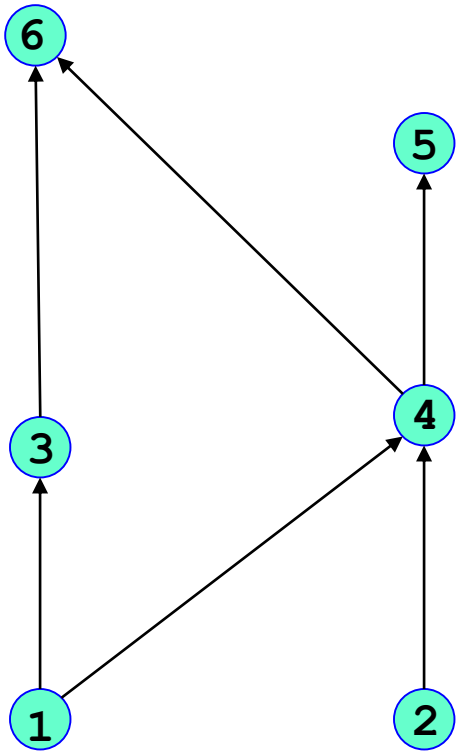
```
S1    a=0;
S2    b=1;
S3    c=a+1
S4    d=b+a;
S5    e=d+1;
S6    f=c+d;
```

Which statements must execute before S_5?

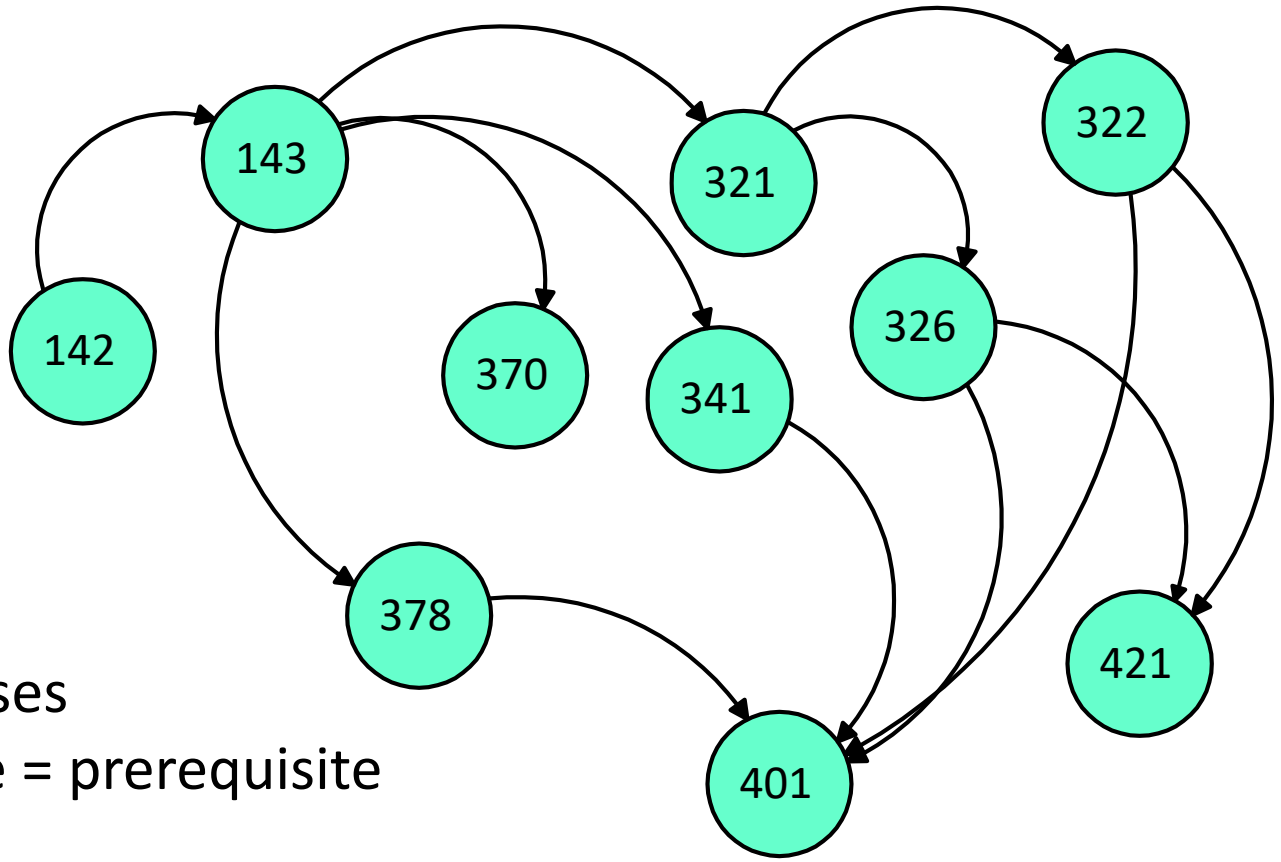
S_1, S_2, S_4

Nodes = statements

Edges = precedence requirements



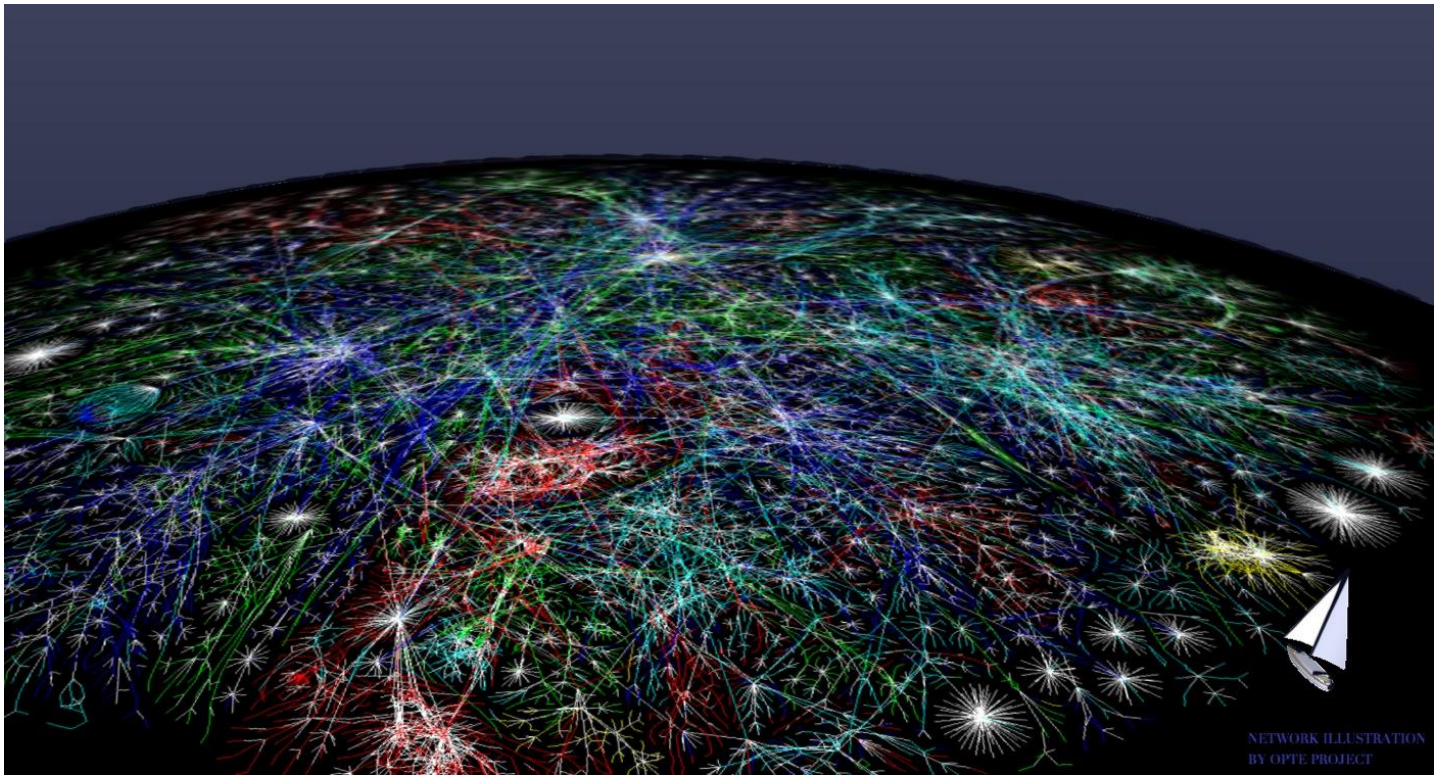
CSE Course Prerequisites at Univ. of Washington

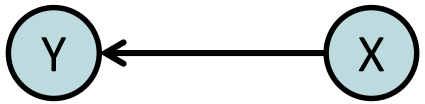


Nodes = courses

Directed edge = prerequisite

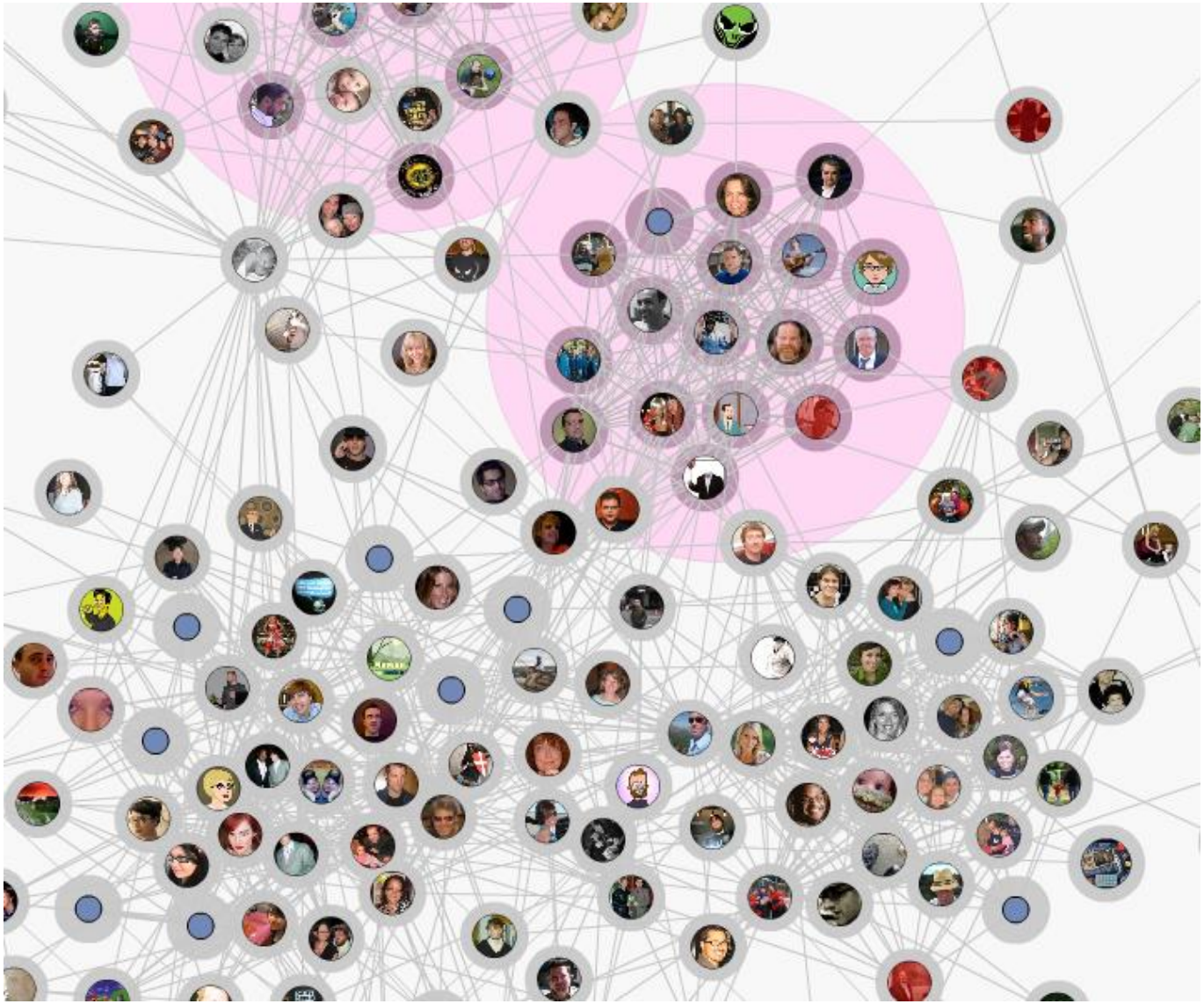
The World Wide Web (Internet)





Nodes = sites
Edges = links

Social Network



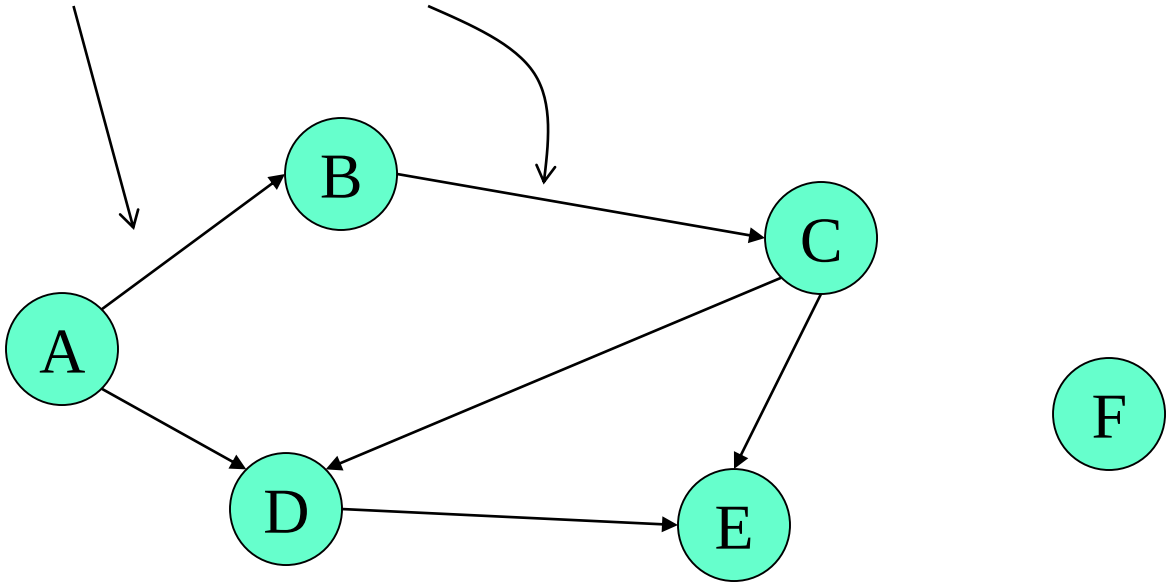
Graph Definition

- A graph is a collection of nodes and edges.

- Formal Definition: A graph *G* is a pair (V, E) where
  - V is a set of vertices or nodes
  - E is a set of edges that connect vertices

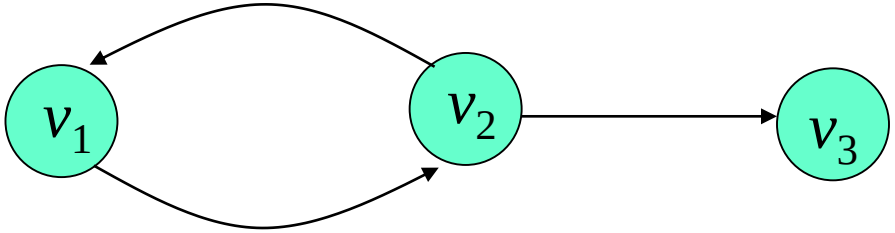
Graph Example

- Here is a directed graph $G = (V, E)$
  - Each edge is a pair (v_1, v_2), where v_1, v_2 are vertices in V
  - $V = \{A, B, C, D, E, F\}$
  - $E = \{(A,B), (A,D), (B,C), (C,D), (C,E), (D,E)\}$

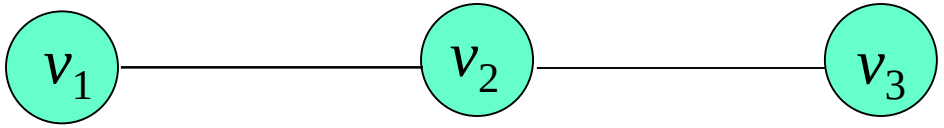


Directed vs Undirected Graphs

- In a directed graph (also called a digraph), the order of nodes in every pair matters. $(v_1, v_2) \neq (v_2, v_1)$

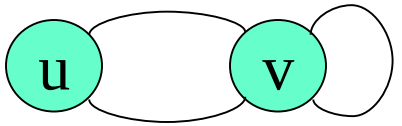


- In an undirected graph $(v_1, v_2) = (v_2, v_1)$. the order of nodes in every pair does not matter.



Undirected Terminology

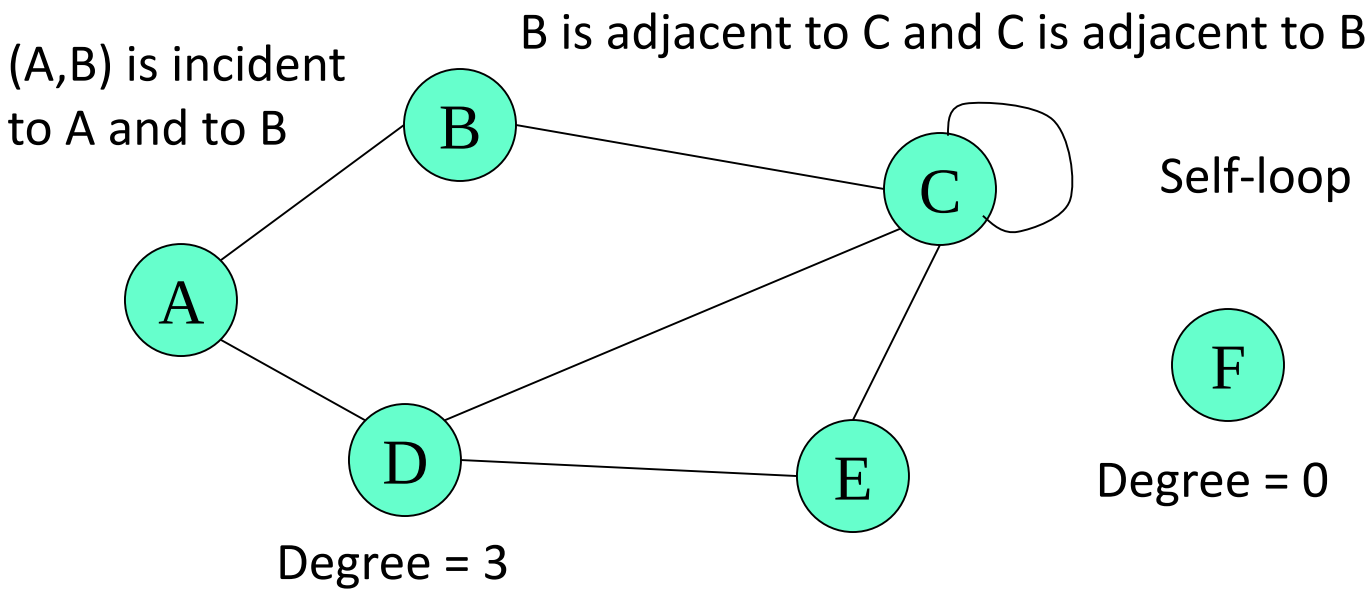
- Two vertices u and v are adjacent in an undirected graph G if (u,v) is an edge in G
  - edge e = (u,v) is incident with vertex u and vertex v
- The degree of a vertex in an undirected graph is the number of edges incident with it
  - a self-loop counts twice (both ends count)
  - denoted deg(v)
- An undirected graph is simple if there are no self-loops and no parallel edges.



self-loop (v,v)

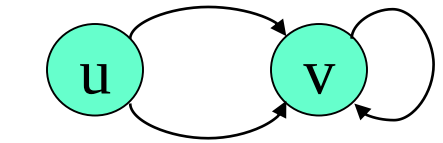
parallel edges (u,v)

Undirected Terminology



Directed Terminology

- In a directed graph, an edge (u,v) goes from u to v.
  - vertex u is the initial (or source) vertex of (u,v)
  - vertex v is the terminal (or target or end) vertex of (u,v)
- Degree:
  - in-degree is the number of edges with the vertex as the terminal vertex
  - out-degree is the number of edges with the vertex as the initial vertex
- A directed graph is simple if there are no self loops and no parallel edges.

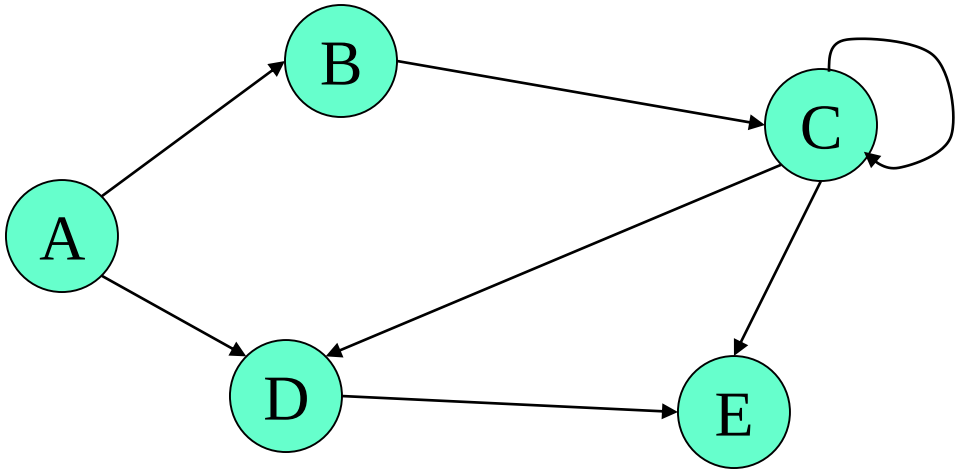


self-loop (v,v)

parallel edges (u,v)

Directed Terminology

(B,C) is an edge from B to C



in-degree = 2
out-degree = 1



in-degree = 0
out-degree = 0

Examples (in class)

- A path: P_n
- A cycle: C_n - defined for $n \geq 3$
- A complete graph (clique): K_n
- A star: S_n

- A bipartite graph: $G=(V_1,V_2,E)$. For every edge $e=(u,v)$, $u \in V_1$, $v \in V_2$.
- A *complete bipartite*, $K_{a,b}$: A bipartite in which $E= V_1 \times V_2$. $|E|=ab$.
- Note: A star is a complete bipartite $K_{1,n}$
- Note: A path is a bipartite.
- What about a cycle?

Handshaking Theorem

- Let G=(V,E) be an undirected graph. Then

$$\sum_{v \in V} \deg(v) = 2 \cdot |E|$$

- Proof: Every edge contributes +1 to the degree of each of the two vertices it is incident with. ∎

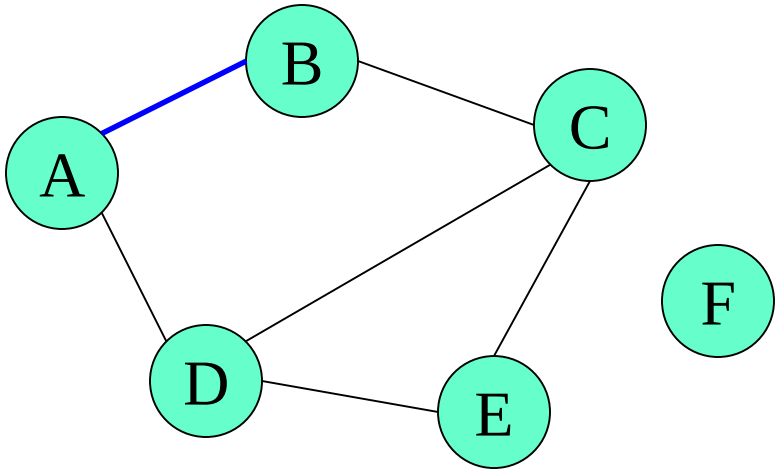
  - number of edges is exactly half the sum of deg(v).
  - Note: the sum of the deg(v) values must be even.

Discussion: What is the importance of this theorem?

Graph Representations

- Space and time are analyzed in terms of:
  - Number of vertices, $n = |V|$ and
  - Number of edges, $m = |E|$
- There are at least two ways of representing graphs:
  - The *adjacency matrix* representation
  - The *adjacency list* representation

Adjacency Matrix

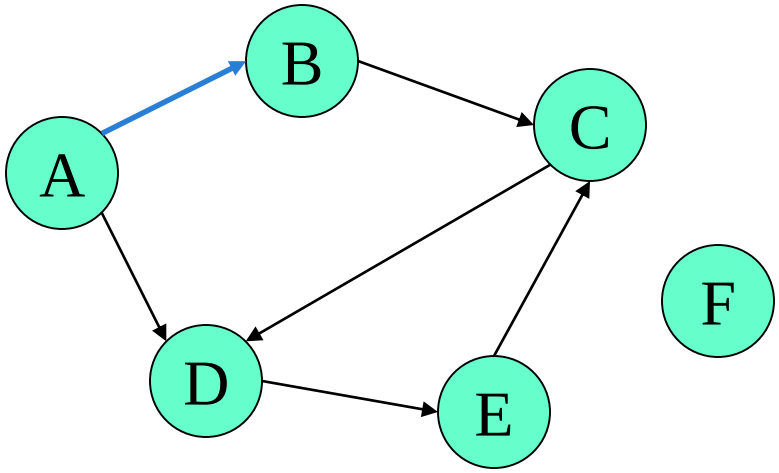


$$M(v, w) = \begin{cases} 1 \text{ if } (v, w) \text{ is in E} \\ 0 \text{ otherwise} \end{cases}$$

	A	B	C	D	E	F
A	0	1	0	1	0	0
B	1	0	1	0	0	0
C	0	1	0	1	1	0
D	1	0	1	0	1	0
E	0	0	1	1	0	0
F	0	0	0	0	0	0

Space = $|V|^2$

Adjacency Matrix for a Digraph



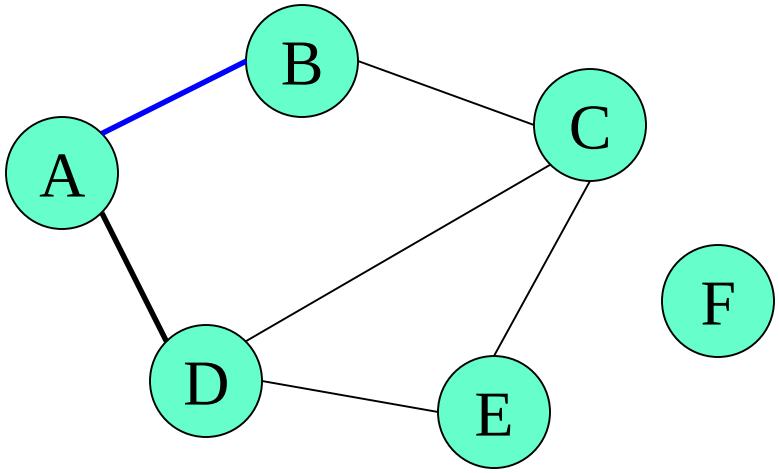
$$M(v, w) = \begin{cases} 1 \text{ if } (v, w) \text{ is in E} \\ 0 \text{ otherwise} \end{cases}$$

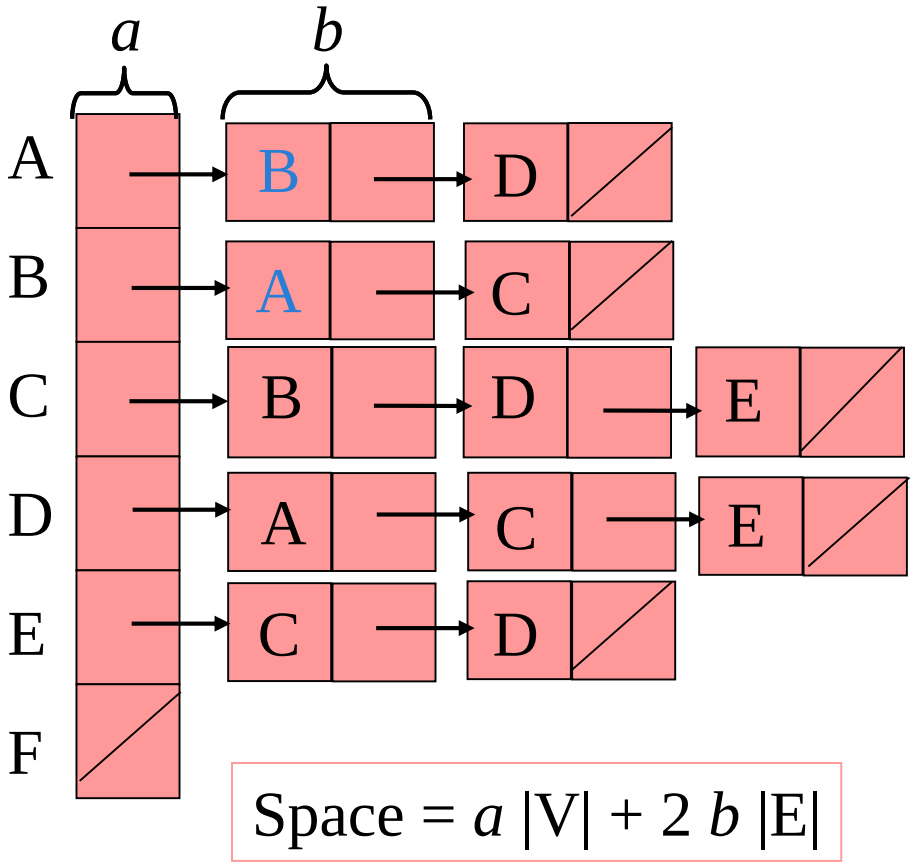
	A	B	C	D	E	F
A	0	1	0	1	0	0
B	0	0	1	0	0	0
C	0	0	0	1	0	0
D	0	0	0	0	1	0
E	0	0	1	0	0	0
F	0	0	0	0	0	0

Space = $|V|^2$

Adjacency List

For each $v \in V$, $L(v)$ = linked list of all w such that (v, w) is in E

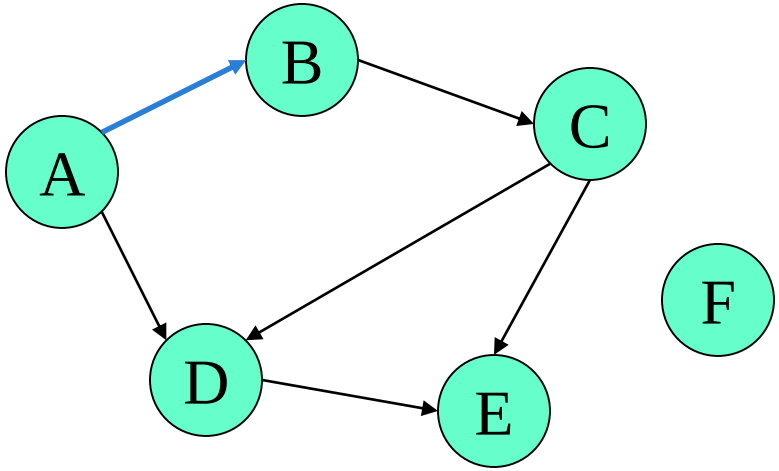


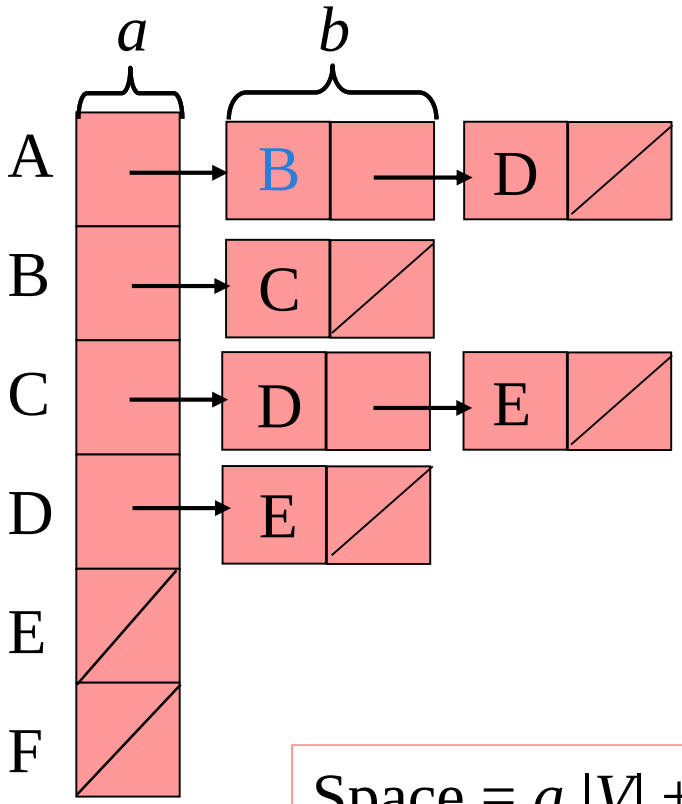


Space = $a\,|V| + 2\,b\,|E|$

Adjacency List for a Digraph

For each $v \in V$, $L(v)$ = linked list of all w such that (v, w) is in E

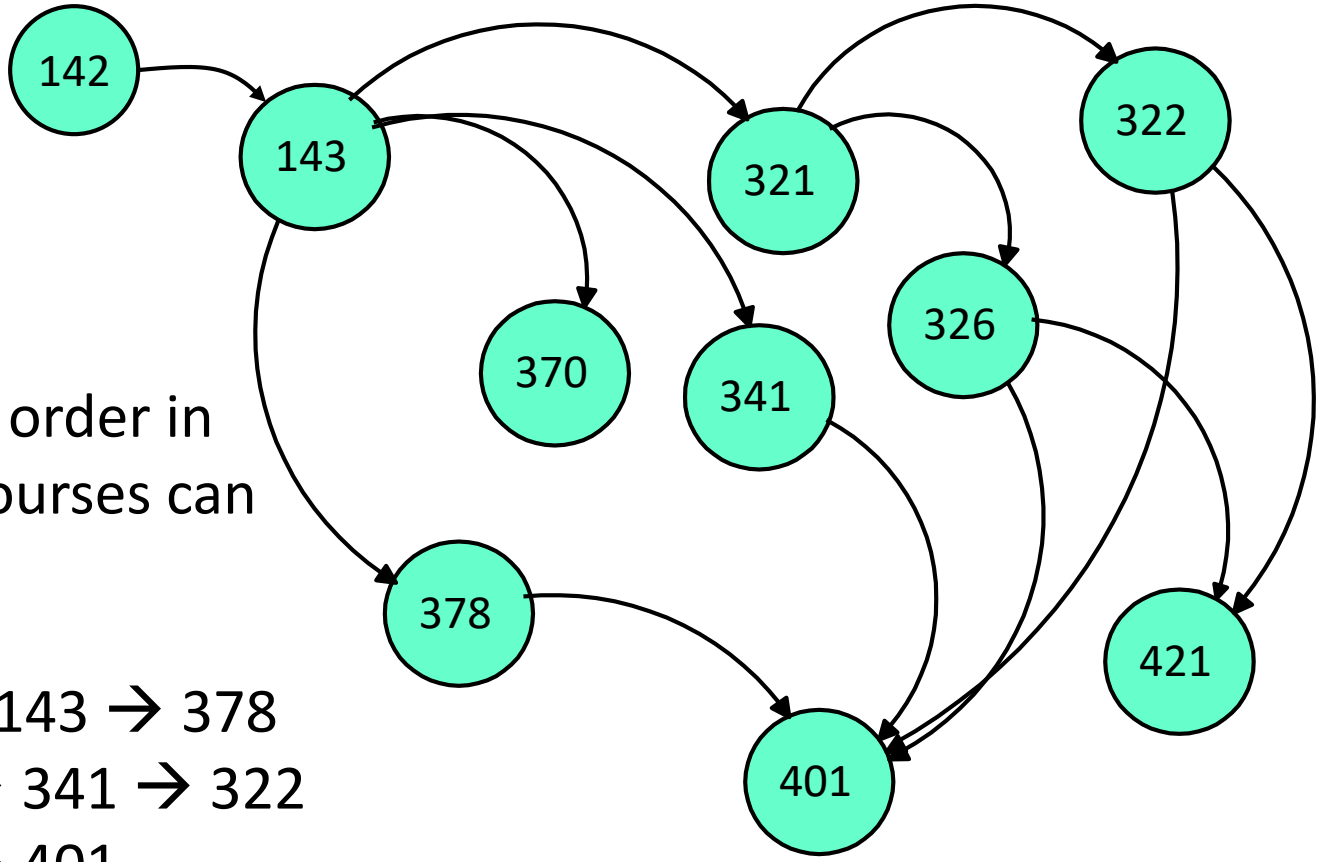




Space = $a\,|V| + b\,|E|$

Topological Sort

In order to take a course, you must take all of its prerequisites first



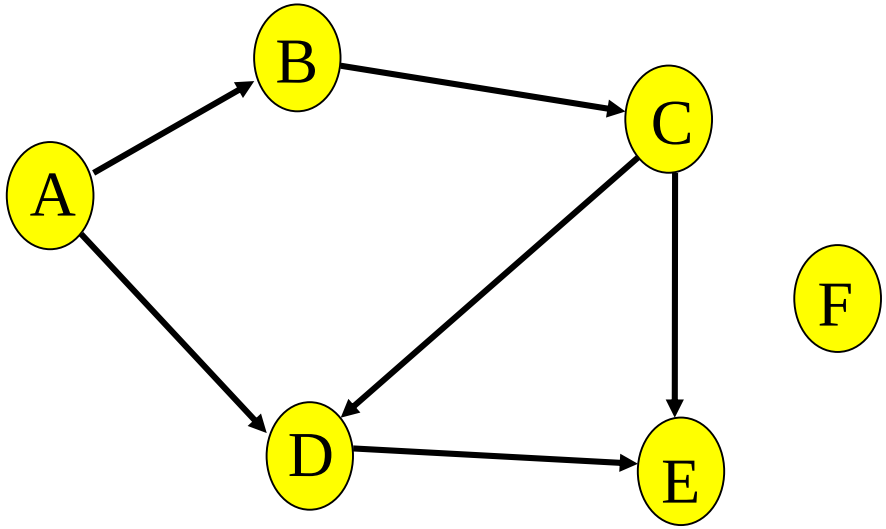
Problem: Find an order in which all these courses can be taken.

Example: 142 → 143 → 378 → 370 → 321 → 341 → 322 → 326 → 421 → 401

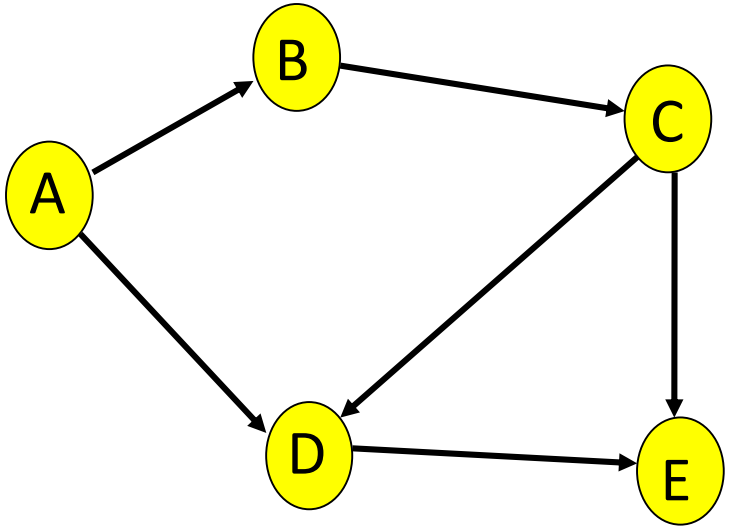
Topological Sort

Given a directed graph $G = (V, E)$, find a linear ordering of its vertices such that:

For any edge $(v,w) \in E$, v precedes w in the ordering

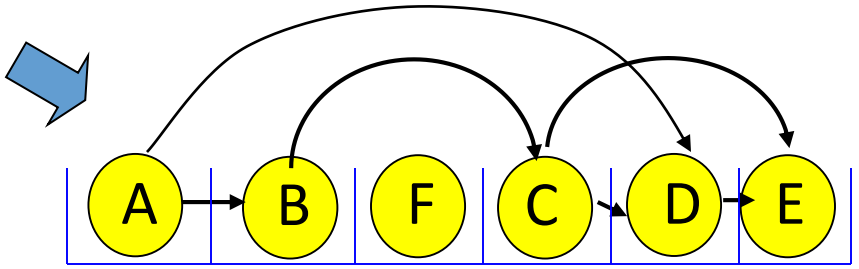


Topo sort - example



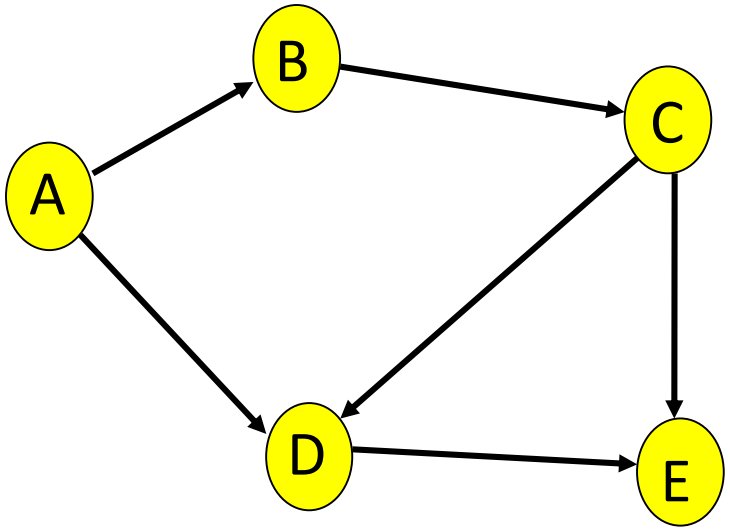
F

Any linear ordering in which all the arrows go to the right is a valid solution



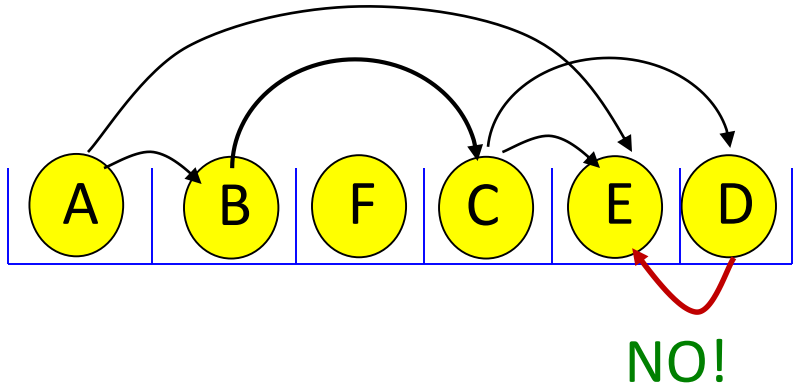
Note that F can go anywhere in this list because it is not connected. In general, the solution is not necessarily unique.

Topo sort - wrong example



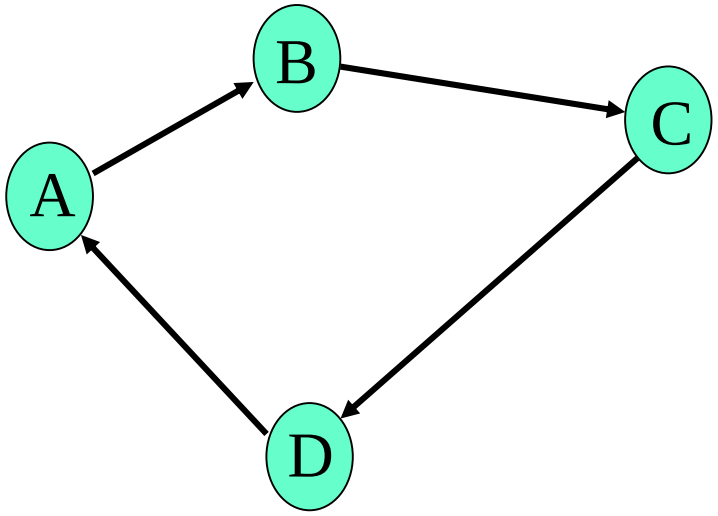
F

Any linear ordering in which an arrow goes to the left is not a valid solution



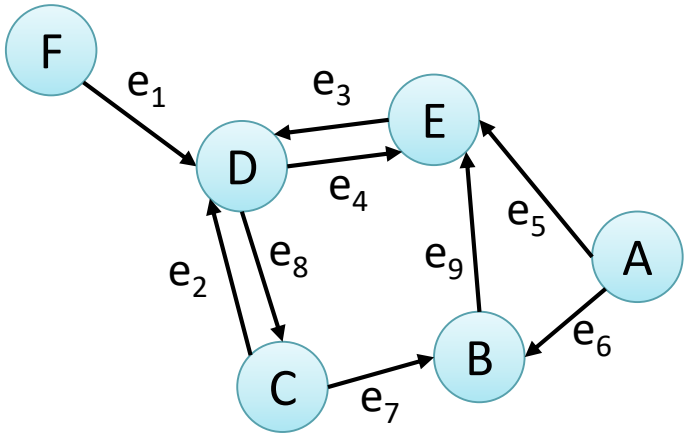
Paths and Cycles

- Given a digraph G = (V,E), a path is a sequence of vertices $v_1, v_2, \ldots, v_k$ such that:
  - $(v_i, v_{i+1}) \in E$ for all $1 \leq i < k$
- A path is a cycle if :
  - $k > 1$ and $v_1 = v_k$
  - Example: A,B,C,D,A
- G is acyclic if it has no cycles.



Paths and Cycles – more examples

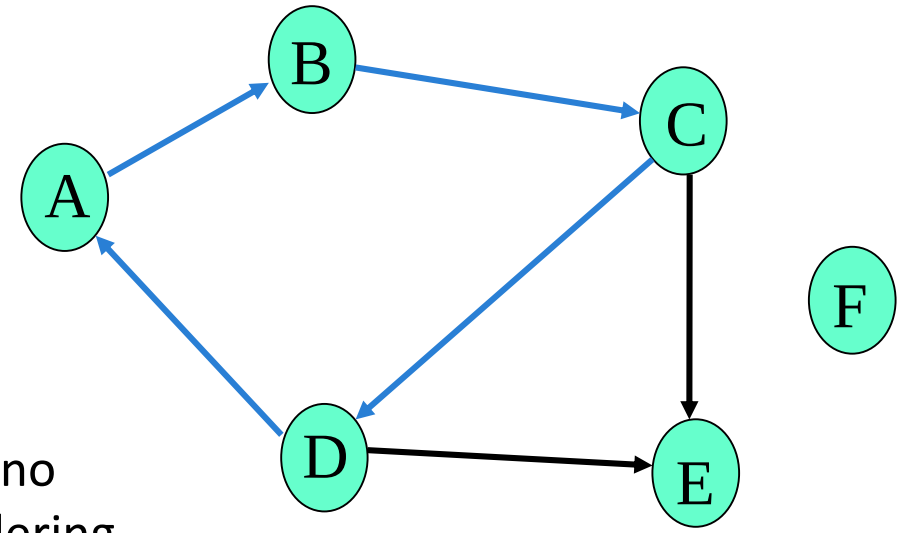
- P_1 = F,D,C,B,E is a path
- P_2 = F,D,C,B,A is not a path
- P_3 = E,D,C,B,E is a cycle



- Paths can be also described by a sequence of edges.
  For example, P_3 can be described as e_3,e_8,e_7,e_9
  Or even this way: $E \xrightarrow{e_1} D \xrightarrow{e_8} C \xrightarrow{e_7} B \xrightarrow{e_9} E$

Only acyclic graphs can be topologically sorted

- A directed graph with a cycle cannot be topologically sorted.

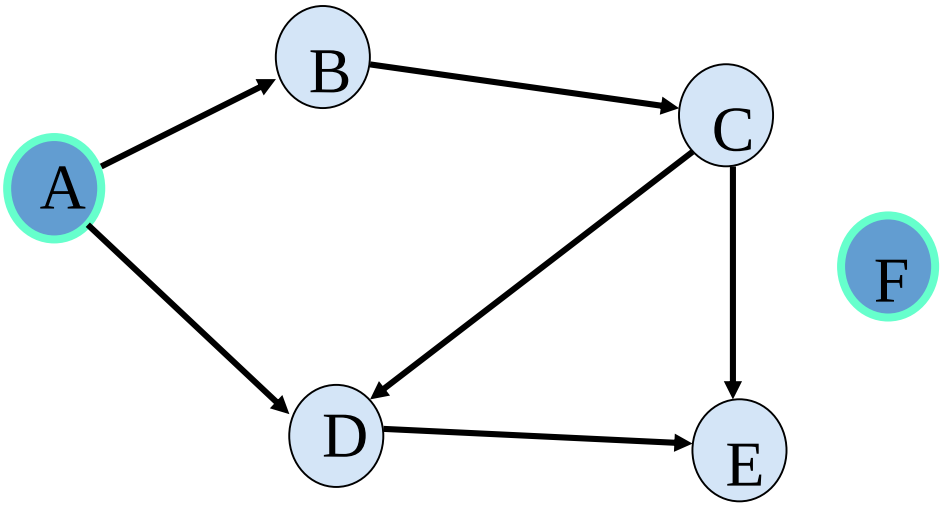


There is no valid ordering of A,B,C,D

Topo sort algorithm - 1

Step 1: Identify vertices that have no incoming edges

- The "in-degree" of these vertices is zero
- Such a vertex is called "a source".

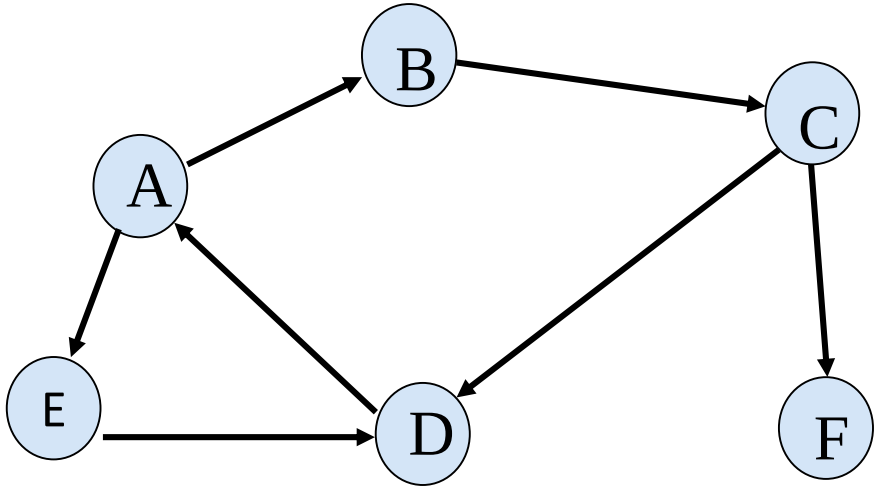


Topo sort algorithm - 1a

Step 1: Identify vertices that have no incoming edges

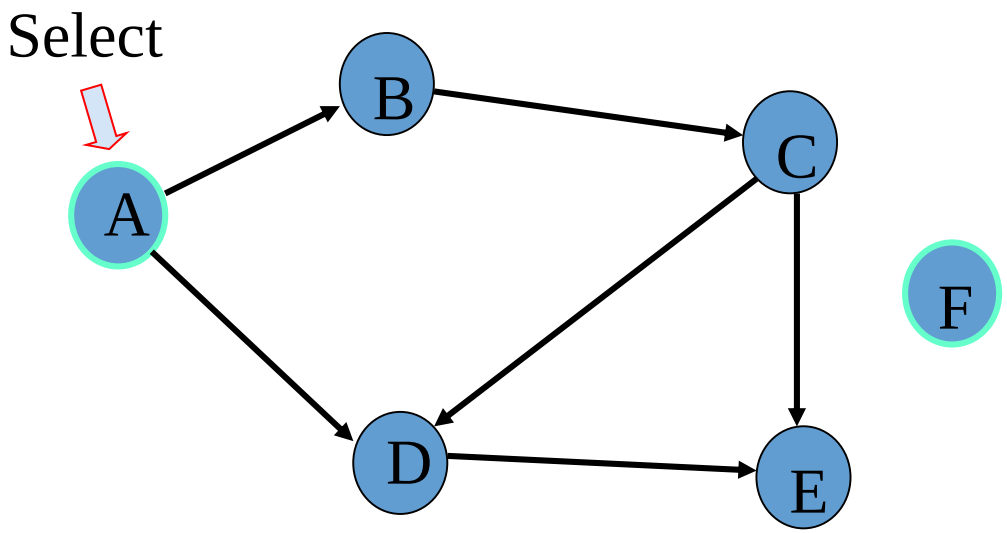
- If *no such vertex exists:*
  - The graph includes a cycle.
  - Topological sort not possible – Halt.

Example of a graph with 'no -source' (not a DAG)



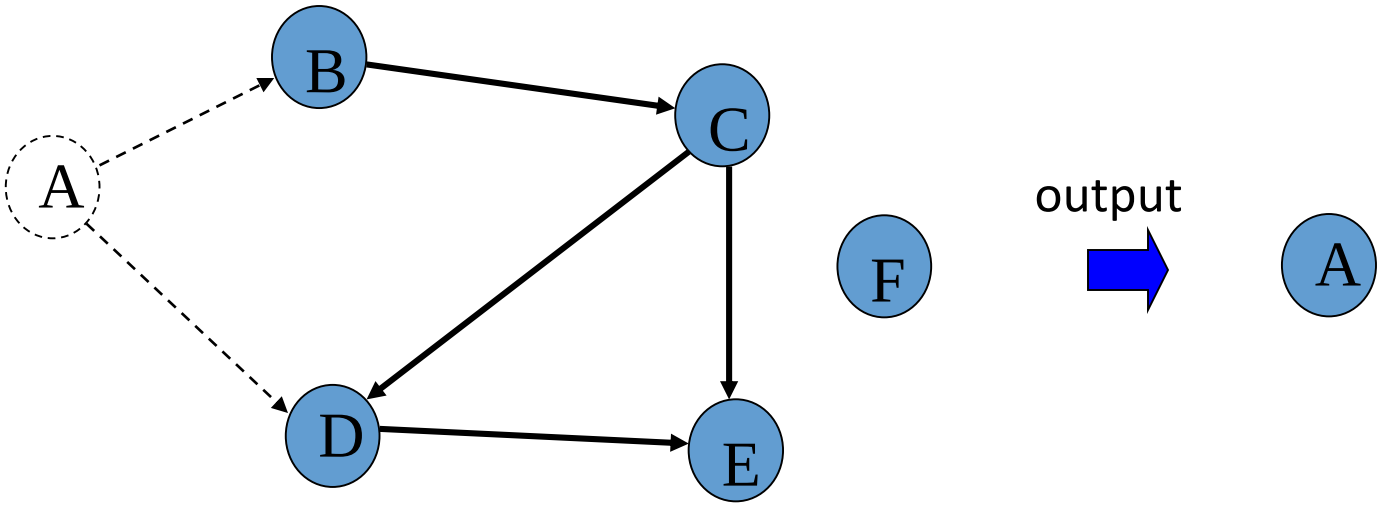
Topo sort algorithm - 1b

Step 1: Select one of the sources



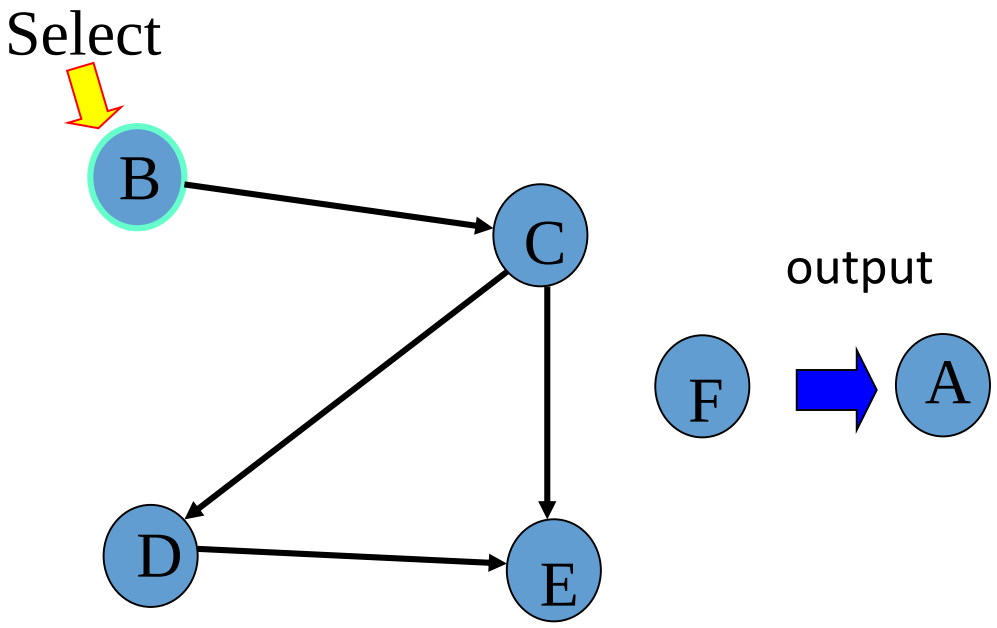
Topo sort algorithm - 2

Step 2: Delete this source and all its outgoing edges from the graph. Place it in the output.



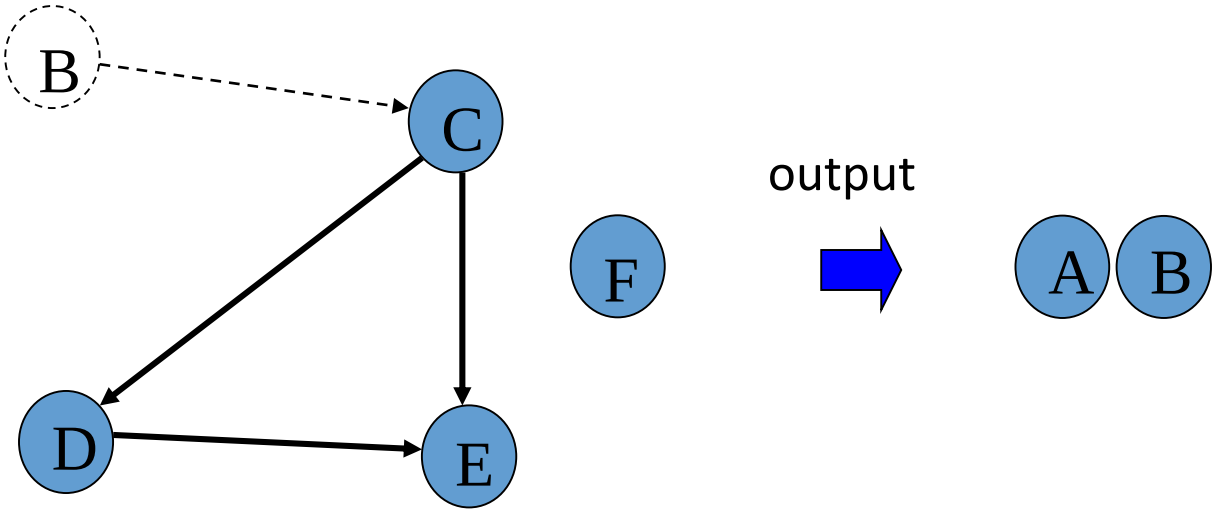
Continue until done

Repeat Step 1 and Step 2 until the graph is empty (or until HALT if not a DAG).



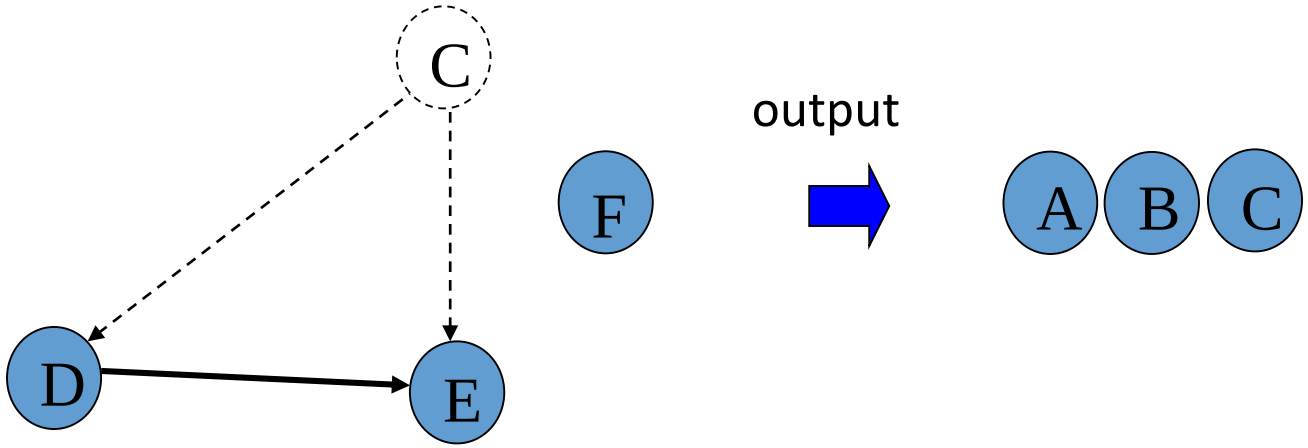
Example (cont') - B

Select B. Add to output. Delete B and its outgoing edges.



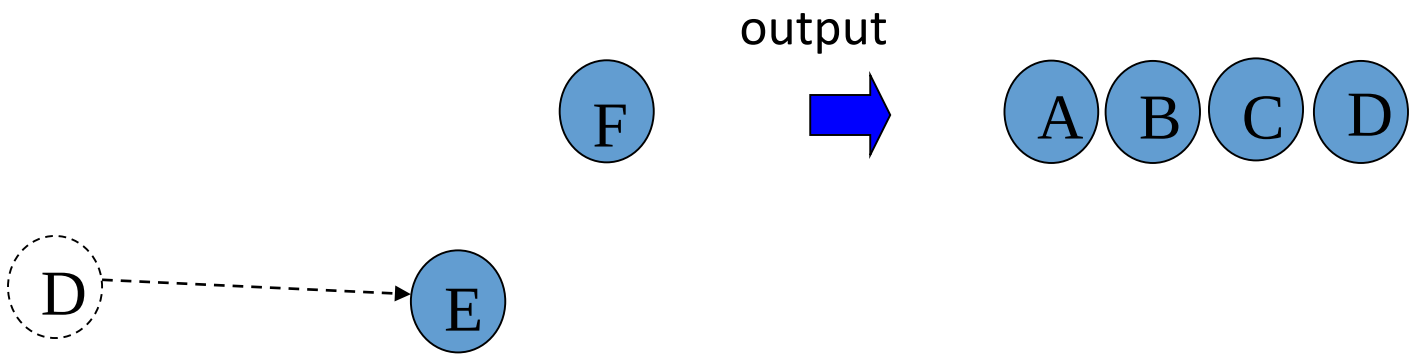
C

Select C. Add to output. Delete C and its outgoing edges.



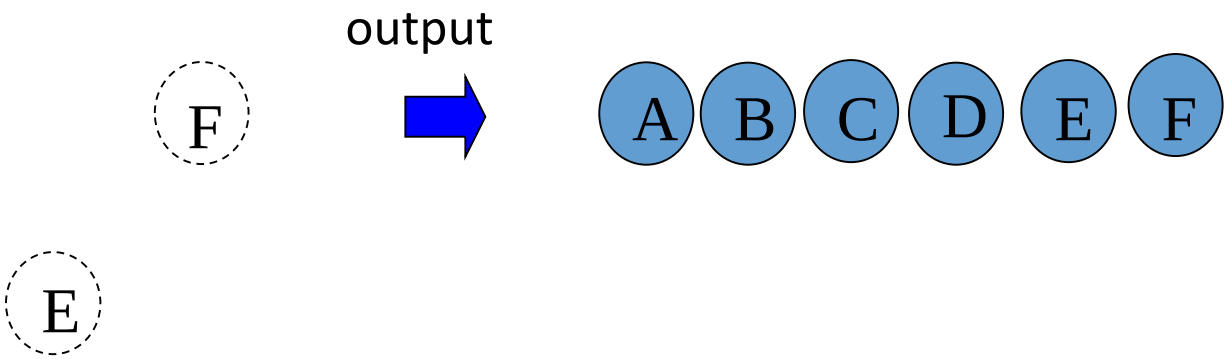
D

Select D. Add to output. Delete D and its outgoing edges.



E, F

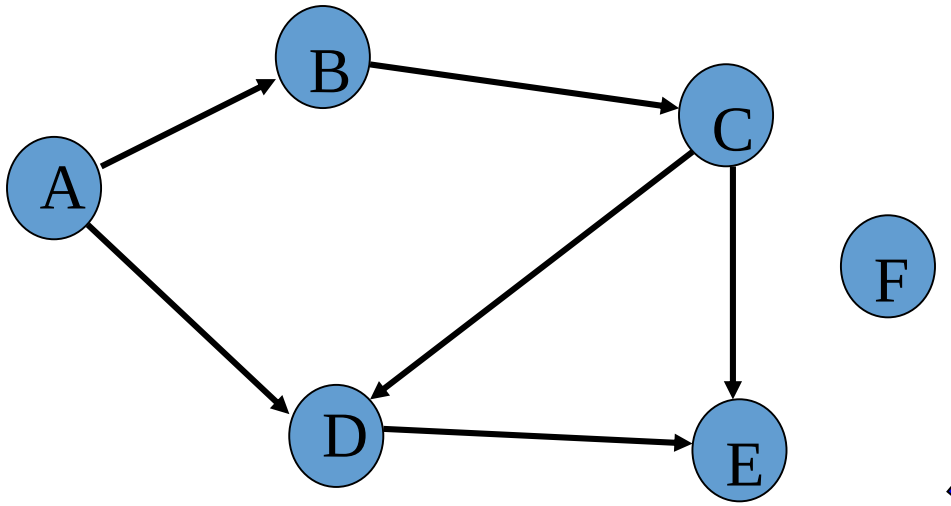
Select E. Add to output. Delete E and its outgoing edges.
Select F. Add to output. Delete F and its outgoing edges.

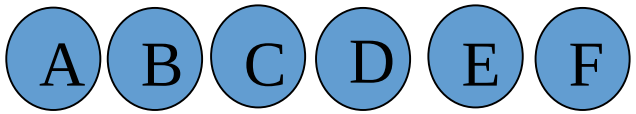


Yes, we could select F earlier (in any step).

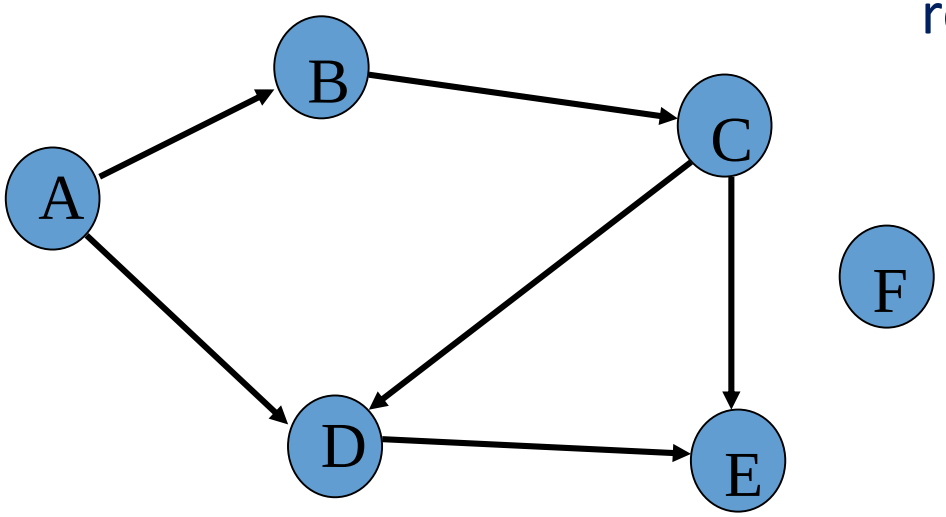
The topological sort is not necessarily unique.

Done





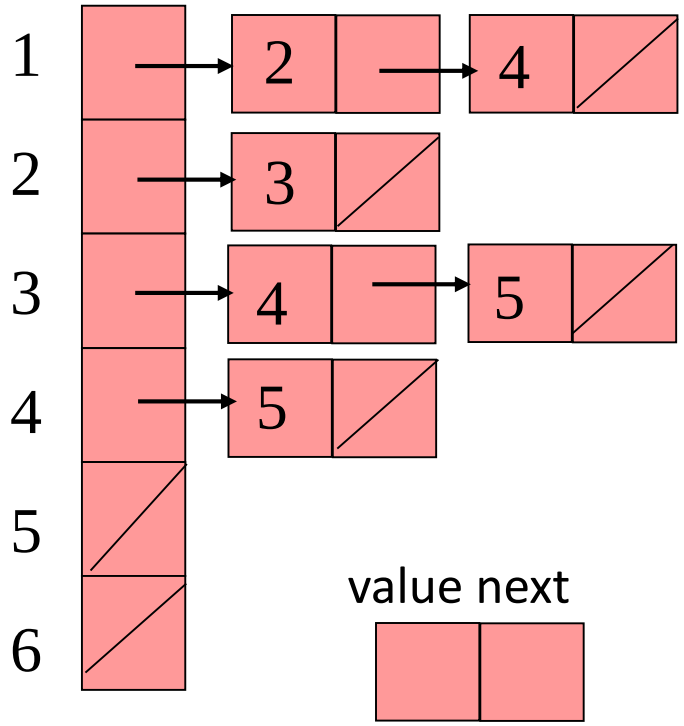
Implementation



Translation array

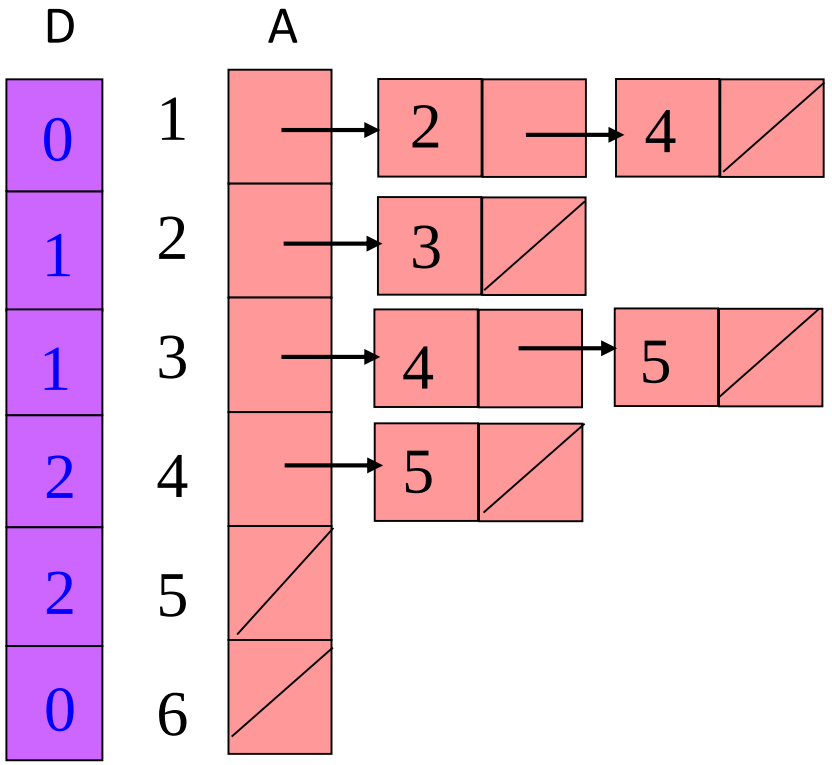
1	2	3	4	5	6
A	B	C	D	E	F

Assume adjacency-list representation



Calculate In-degrees

In-Degree array; or add a field to array A



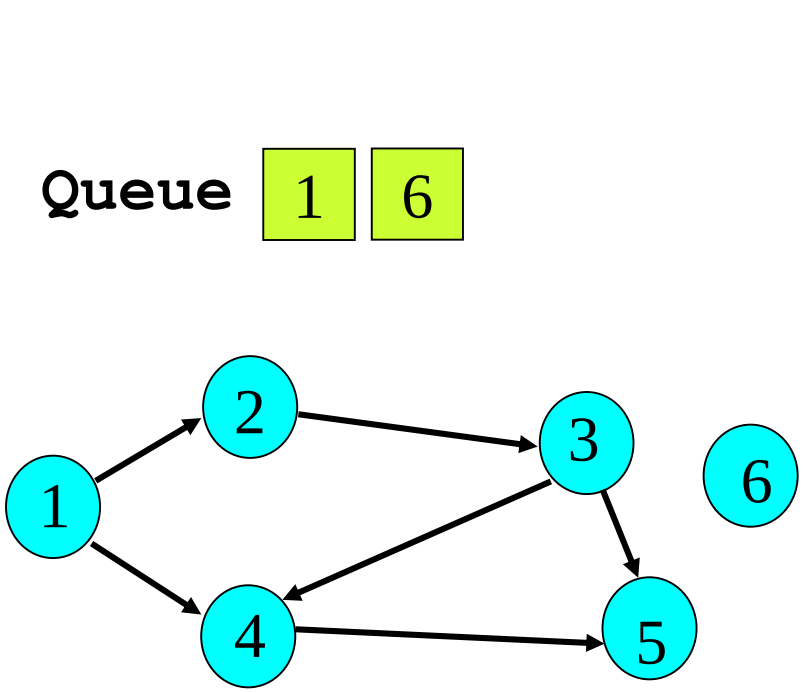
Calculate In-degrees

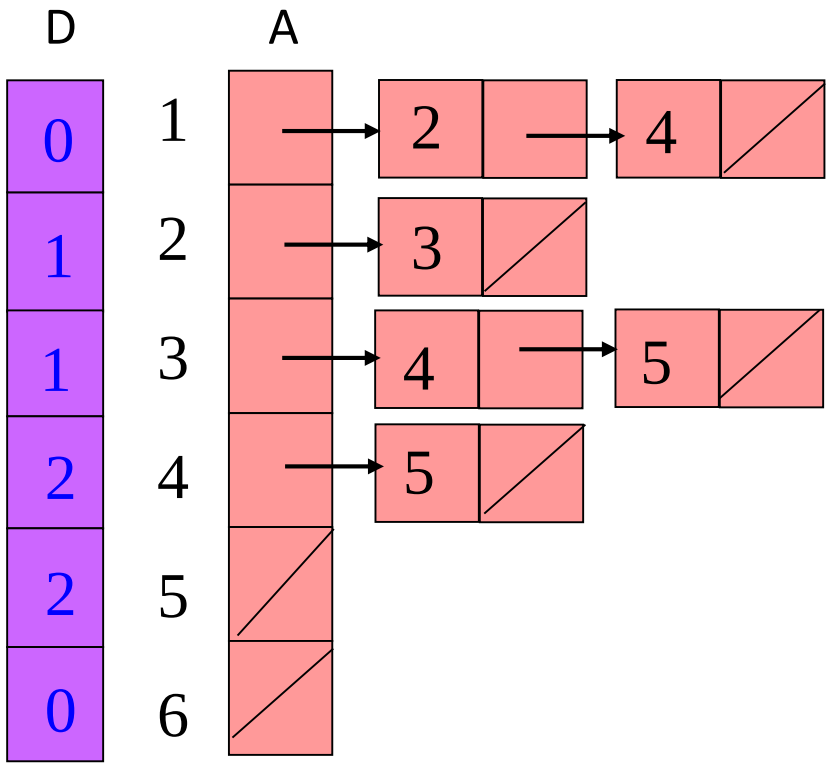
```
for i = 1 to n do D[i] := 0; endfor
for i = 1 to n do
  x := A[i];
  while x ≠ null do
    D[x.value] := D[x.value] + 1;
    x := x.next;
  endwhile
endfor
```

Time Complexity? O(n+m).

Maintaining Degree-0 Vertices

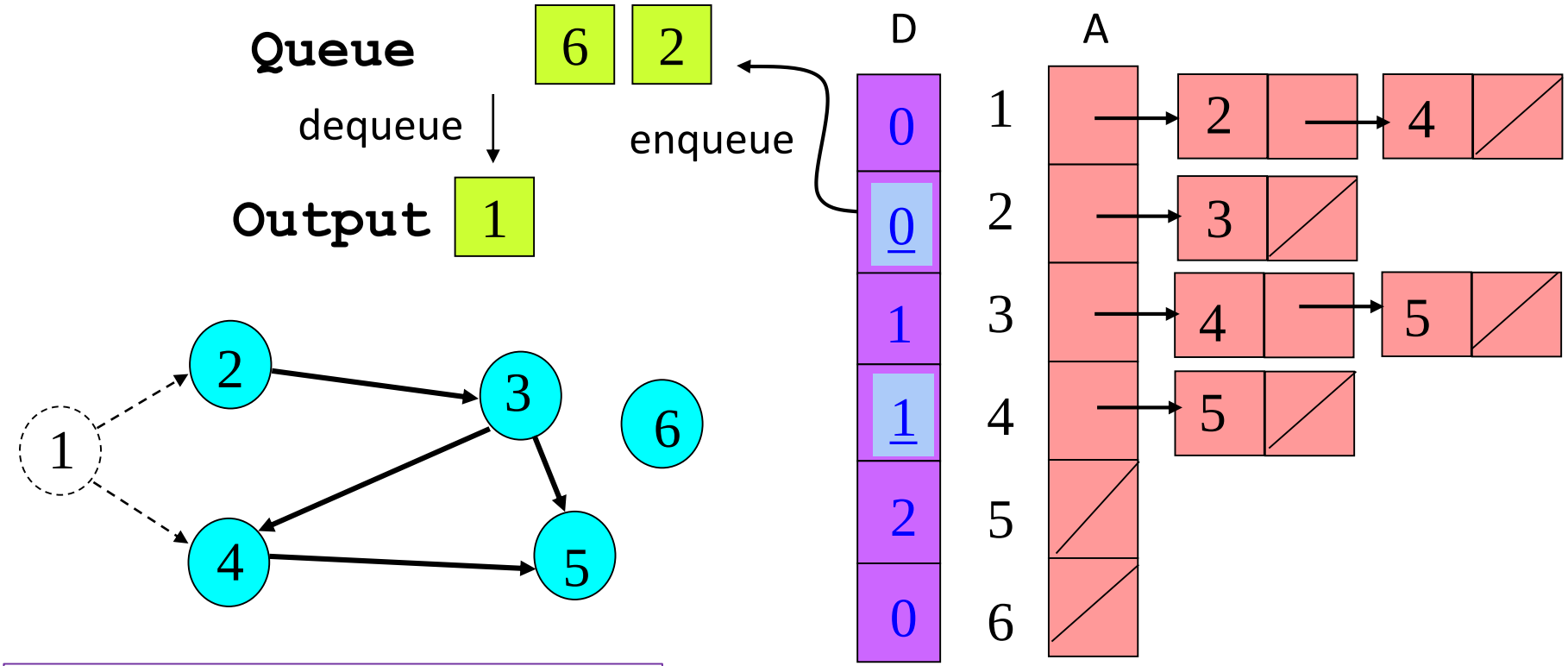
Key idea: Initialize and maintain a *queue (or stack)* of sources.





Topo-Sort using a Queue

After each vertex is output, when updating In-Degree array, *enqueue any vertex whose In-Degree becomes zero*



Output with a queue: 1,6,2,3,4,5

Topological Sort Algorithm

1. Store each vertex's In-Degree in an array D
2. Initialize queue with all source vertices (in-deg(v)=0)
3. While there are vertices remaining in the queue:
   (a) Dequeue and output a vertex
   (b) Reduce In-Degree of all vertices adjacent to it by 1
   (c) Enqueue any of these vertices whose In-Degree became zero
4. If all vertices are output then success, otherwise there is a cycle.

Some Detail





```
Main Loop
while notEmpty(Q) do
   x := Dequeue(Q)
   Output(x)
   y := A[x];
   while y ≠ null do
      D[y.value] := D[y.value] - 1;
      if D[y.value] = 0 then Enqueue(Q,y.value);
      y := y.next;
   endwhile
endwhile
```

Time complexity? O(out_degree(x)) .

Topological Sort Analysis

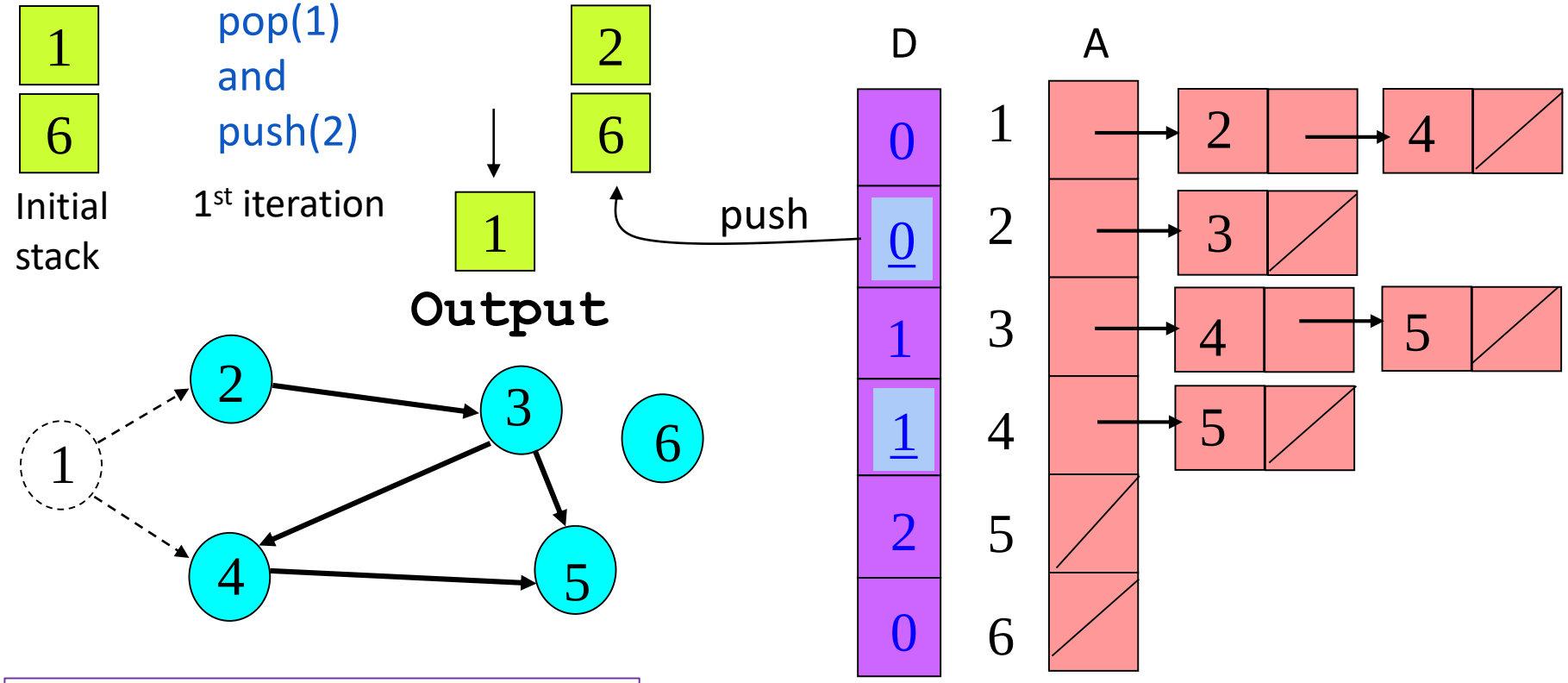
- Initialize In-Degree array: O(|V| + |E|)
- Initialize Queue with source vertices: O(|V|)
- Dequeue and output vertex:
  - |V| vertices, each takes only O(1) to dequeue and output: O(|V|)
- Reduce In-Degree of all vertices adjacent to a vertex and Enqueue any In-Degree 0 vertices:
  - O(|E|) (total out_degree of all vertices)
- For input graph G=(V,E) total run time = O(|V|+|E|)
  - Linear time!

Topo-Sort using a Stack

After each vertex is output, when updating In-Degree array, *push any vertex whose In-Degree becomes zero.*

In the example, assume we push(6) and then push(1).

When node 2 becomes ready it is pushed above the 6.



Output with a stack: 1,2,3,4,5,6